Containerisation: Docker, Podman, and How Kubernetes Fits In

Mick Cooney

12 October 2026

Table of Contents

1 Introduction **3**
- 1.1 What This Covers . . . 4
- 1.2 What This Is Not . . . 5
- 1.3 Assumed Background . . . 5
- 1.4 How the Guide Is Structured . . . 5

2 What a Container Actually Is **6**
- 2.1 The Lineage: From chroot Onwards . . . 6
- 2.2 Namespaces: Controlling What a Process Can See . . . 7
- 2.3 Control Groups: Controlling What a Process Can Use . . . 8
- 2.4 Union Filesystems and the Image Layer Model . . . 9
- 2.5 Container Versus Virtual Machine . . . 9

3 The Practical Container Contract **10**
- 3.1 Building and Identifying Images . . . 10
- 3.2 Storage and Lifecycle . . . 11
- 3.3 Networking Is a Separate Layer . . . 11
- 3.4 Security Beyond Namespaces . . . 12

4 Standardisation: The OCI Specifications **12**

5 Docker **13**
- 5.1 Origins and What Docker Actually Invented . . . 14
- 5.2 Architecture: CLI, dockerd, containerd, runc . . . 14
- 5.3 Licensing and Commercial Status in 2026 . . . 15
- 5.4 Docker Compose . . . 16
- 5.5 Docker Swarm . . . 16
- 5.6 Docker Hub . . . 17

6 Podman **18**
- 6.1 Daemonless Architecture . . . 18
- 6.2 Rootless by Default, and Why It Matters . . . 18

6.3 systemd Integration and Quadlet . . . 19
6.4 Compose, Desktop, and Kubernetes YAML . . . 20
6.5 Podman's "Pod" Concept . . . 21
6.6 Buildah and Skopeo . . . 21
6.7 Governance, Versions, and the CNCF Sandbox . . . 22
6.8 Adoption Reality . . . 22

7 Docker Versus Podman in 2026 **23**
7.1 The Technical Differences That Remain . . . 23
7.2 When to Pick Docker . . . 24
7.3 When to Pick Podman . . . 24
7.4 The Honest Summary . . . 25

8 The Wider Runtime Landscape **25**
8.1 containerd . . . 26
8.2 CRI-O . . . 26
8.3 runc and crun . . . 27
8.4 gVisor . . . 27
8.5 Kata Containers . . . 28
8.6 Choosing Between Them, and RuntimeClass . . . 28
8.7 LXC and LXD . . . 29
8.8 WebAssembly and WASI . . . 29
8.9 Firecracker and the microVM Model . . . 31

9 Use Cases: From One Container to a Platform **31**
9.1 Reproducible Development and CI . . . 32
9.2 Batch Data Work . . . 33
9.3 Notebooks and Interactive Data Science . . . 33
9.4 Machine Learning Training . . . 34
9.5 Model Serving and Inference . . . 34
9.6 When the Ladder Reaches Kubernetes . . . 35

10 Why Kubernetes Exists **35**
10.1 Scheduling Across a Fleet . . . 35
10.2 Self-Healing . . . 36
10.3 Declarative Reconciliation . . . 36
10.4 Service Discovery and Stable Networking . . . 37
10.5 Rolling Updates and Rollbacks . . . 37
10.6 Scaling . . . 37
10.7 Multi-Tenancy and Resource Governance . . . 38
10.8 The Honest Counter-Case . . . 38

11 Kubernetes Core Architecture **38**
11.1 kube-apiserver . . . 39
11.2 etcd . . . 39
11.3 kube-scheduler . . . 40

11.4 kube-controller-manager . . . 41
11.5 kubelet . . . 41
11.6 kube-proxy and the Networking Model . . . 42

12 The Kubernetes Object Model **42**
12.1 Pod . . . 42
12.2 ReplicaSet and Deployment . . . 43
12.3 Service . . . 43
12.4 Ingress and the Gateway API . . . 44
12.5 ConfigMap and Secret . . . 44
12.6 Namespace . . . 45
12.7 StatefulSet . . . 45
12.8 DaemonSet . . . 46
12.9 Job and CronJob . . . 46
12.10 PersistentVolume and PersistentVolumeClaim . . . 46

13 The Kubernetes Ecosystem **47**
13.1 Helm . . . 47
13.2 Kustomize . . . 47
13.3 Custom Resources and the Operator Pattern . . . 48
13.4 Policy and Admission . . . 48
13.5 Observability and Debugging . . . 49
13.6 Service Meshes and the Ambient Shift . . . 49
13.7 GitOps: Argo CD and Flux . . . 50

14 Lightweight Kubernetes Distributions **51**

15 Alternatives to Kubernetes **52**

16 Where Things Stand in 2026 **53**
16.1 Release Cadence and Versions . . . 53
16.2 GPU Scheduling and AI Workloads . . . 53
16.3 Supply-Chain Security . . . 54
16.4 Ecosystem Scale and the Wasm Correction . . . 55

17 Closing: A Usable Mental Model **55**

18 Further Reading **57**

1 Introduction

Containers are the substrate almost everything you deploy now runs on, and most practitioners learn them by osmosis — a `Dockerfile` inherited from a colleague, a `docker compose up` in a README, a YAML manifest someone in platform engineering wrote two years ago and nobody has touched since. That works right up until it doesn't: until a build behaves

differently in CI than on your laptop, until a rootless container can't write to a mounted volume, until a Pod sits in `Pending` forever and the only diagnostic you have is a vague memory that Kubernetes "schedules things". At that point the gap between using containers and understanding them turns into wasted days.

This primer closes that gap. It is written for the engineer or data scientist who ships containerised software, reads Kubernetes manifests, and has a working intuition for what a container does — but has never been shown the machinery underneath, and so cannot reason about it when it misbehaves. The goal is not to make you a platform engineer. It is to give you an accurate mental model: what a container actually is at the kernel level, how the user-facing engines and lower-level runtimes fit together, why Kubernetes exists at all, and what the 2026 landscape looks like now rather than what it looked like when the blog post you read was written.

1.1 What This Covers

The primer works bottom-up. It starts with the Linux kernel primitives that make containers possible — namespaces, control groups, and union filesystems — and the lineage that produced them, so that "container" stops being a black box and becomes a specific, describable arrangement of ordinary kernel features. From there it covers the Open Container Initiative specifications, which are the reason the rest of the ecosystem is interchangeable rather than a set of walled gardens.

It then treats the two engines a practitioner actually chooses between. Docker gets its history, its layered client/daemon/`containerd`/`runc` architecture, its current licensing position — which in 2026 is a live commercial concern rather than a footnote — and the state of Compose, Swarm, and Docker Hub. Podman gets its daemonless and rootless architecture, its `systemd` integration, its companion tools, and its recent governance shift. The two then get compared honestly, on the axes that actually decide the choice in 2026.

Next comes the practical container contract — image identity and builds, storage, networking, and the security mechanisms that sit alongside namespaces and cgroups — followed by the wider runtime landscape: `containerd` and CRI-O, `runc` and `crun`, the hardened runtimes gVisor and Kata Containers, LXC/LXD's quite different model of what a container is for, WebAssembly as a complementary deployment target rather than a successor, and Firecracker microVMs. A short section on use cases then draws the line between what a container solves on its own and where an orchestrator like Kubernetes actually becomes necessary — the bridge into the primer's second half.

The second half of the primer is Kubernetes. It starts with the coordination problems Kuber-

netes exists to solve, because Kubernetes is incomprehensible until you know what it is for. Then the architecture — control plane and node components — then the object model, explained by purpose rather than by definition. Then the ecosystem that has grown on top of it: Helm, Kustomize, Operators, service meshes, GitOps, and ingress. Then the lightweight distributions, the genuine alternatives to Kubernetes, and finally the developments from 2025 and 2026 a practitioner should know about — GPU scheduling for AI workloads above all.

1.2 What This Is Not

This is not a tutorial. There are no step-by-step installation walkthroughs, no "build your first container" exercises, and no command reference — the official documentation does those well and keeps them current in a way a primer cannot. Commands appear here only where they illustrate a point about how something works.

It is not a Kubernetes operations manual. Cluster upgrades, etcd backup and restore, network plugin selection, capacity planning, and incident response are real disciplines that this primer only points at. Nor is it a security hardening guide: the security discussion here explains where the boundaries actually are and why, which is the prerequisite for hardening, but it stops short of a control checklist.

It is also not neutral where neutrality would be dishonest. Where one option is better for a stated purpose, the primer says so and gives the reasoning. Where the evidence is soft — adoption surveys, vendor-published performance numbers, market-share claims — it says that too, rather than laundering marketing figures into facts.

1.3 Assumed Background

You should be comfortable on a Unix command line, know what a process is, and have a rough sense of what a filesystem mount is. You should have used containers at least a little, even if only by running commands someone else wrote. Familiarity with the difference between a build and a runtime helps. You do not need any prior Kubernetes knowledge, any kernel development experience, or any distributed-systems theory. Where a concept from those areas is load-bearing, it gets introduced where it is used.

1.4 How the Guide Is Structured

The order is deliberate: kernel mechanics, then standards, then engines, then orchestration, then the current state of play. Each section assumes the ones before it. If you are only here for Kubernetes, you can start at the section on why Kubernetes exists — but the architecture discussion will make considerably more sense if you have read the OCI section

first, because Kubernetes' relationship to container runtimes is defined entirely in terms of those specifications.

Verified against primary sources for all sections; unverified claims are flagged inline.

2 What a Container Actually Is

A container is a process. That is the single most useful sentence in this primer. When you run a container on a Linux host, the host kernel starts an ordinary process — you can see it in `ps` on the host, kill it with `kill`, and read its memory maps in `/proc`. There is no guest operating system, no virtual hardware, no second kernel. What makes it a *container* rather than just a process is that the kernel has been asked to lie to it about what it can see, constrain what it can use, and give it a different root filesystem to look at.

Those three things map onto three kernel features: namespaces control what a process can see, control groups control what it can consume, and a union filesystem provides the root filesystem cheaply. Everything else in the container world — Docker, Podman, Kubernetes, the whole registry ecosystem — is tooling built on top of that trio.

2.1 The Lineage: From chroot Onwards

The oldest ancestor is `chroot`, which arrived in Version 7 Unix in 1979 and changed a process's idea of `/`. Run a process under `chroot /some/directory` and its filesystem root becomes that directory; it cannot name paths outside it. This is filesystem isolation and nothing else. The process still shares the host's process table, network stack, hostname, user list, and every resource limit. `chroot` was never a security boundary and should not be treated as one — a process with root privileges inside a `chroot` has a small library of well-documented escape techniques, most of them involving opening a directory file descriptor before the call and using it afterwards. It matters historically because it established the idea: give a process a different view of the system, and you can run software that believes it owns the machine.

FreeBSD jails (2000) extended that idea properly by also partitioning the process table and network interfaces, so a jailed process could not see or signal processes outside the jail. Solaris Zones (2004) went further still, adding resource controls and a coherent administrative model. Both were genuine isolation mechanisms, and both were confined to their operating systems.

Linux got there by a longer and more piecemeal route. Namespaces arrived one at a time from 2002 onwards, each addressing a different global resource. Control groups were merged in 2008, contributed by Google, who needed resource accounting and limits for the enormous number of processes they were packing onto individual machines. LXC (2008) was the first

tool to wire namespaces and cgroups together into something resembling a container manager, and it worked — but it exposed the kernel's model more or less directly, which meant using it required understanding all of it.

Docker's contribution in 2013 was not a new isolation mechanism. It was packaging: a single CLI, a declarative build file, a layered image format, and a public registry, wrapping primitives that had existed for years. This explains why the container ecosystem fragmented as it did. The hard part was never the kernel; it was the developer experience and the distribution format. Once those were standardised, the engines became interchangeable.

2.2 Namespaces: Controlling What a Process Can See

A namespace wraps a global system resource so that processes inside the namespace see their own instance of it. Linux has several, and a typical container uses most of them at once:

- **PID (process ID) namespace** — the process gets its own process-ID tree. The first process in the namespace is PID 1, and processes outside the namespace are not visible in `/proc`. This is why `ps` inside a container shows two or three processes rather than the host's several hundred, and it is also why a container's main process inherits PID 1's special responsibilities, including reaping orphaned child processes, which most application binaries were never written to do.
- **Network namespace** — its own network interfaces, routing table, `iptables` rules, and socket ports. Two containers can each bind port 8080 without conflict because each has a private port space. Connectivity between namespaces is then arranged explicitly, usually with a virtual ethernet pair bridging the container namespace to the host's.
- **Mount namespace** — its own mount table. This is what makes the container's root filesystem possible without disturbing the host's, and what makes bind-mounting a host directory into a container a per-container operation.
- **UTS (Unix Timesharing System) namespace** — its own hostname and domain name, which is why `hostname` inside a container returns a container ID rather than your laptop's name.
- **IPC (inter-process communication) namespace** — its own System V IPC objects and POSIX message queues, so shared-memory segments don't leak between containers.
- **User namespace** — its own user and group ID mapping. This is the most consequential one for security, and it is discussed in detail in the Podman section, because it is the mechanism that makes rootless containers work.
- **Cgroup namespace** — its own view of the control-group hierarchy, so a container can't see the resource limits applied to its siblings.

The critical property is that namespaces are per-process and composable. You are not creating a "container object" that the kernel tracks; you are calling `clone()` with a set of flags, or `unshare()` and `setns()` on an existing process, and the kernel gives that process a different view. A process can be in a new network namespace but share the host's PID namespace, or any other combination. Container engines pick a conventional set; nothing forces that convention.

This composability is also the source of a persistent class of confusion. Kubernetes Pods share a network namespace but not a mount namespace, which is why containers in a Pod reach each other on `localhost` but cannot see each other's files without an explicitly shared volume. Once you know namespaces are independent, that stops being a special rule to memorise and becomes an obvious consequence.

2.3 Control Groups: Controlling What a Process Can Use

Namespaces isolate visibility; they do nothing about consumption. A process in its own namespaces can still allocate all the host's memory, saturate every CPU, and fork until the process table is exhausted. Control groups — cgroups — are the accounting and limiting mechanism that closes that gap. You arrange processes into a hierarchy and attach controllers for CPU time, memory, block I/O, and process count to nodes in that hierarchy.

Two generations exist. cgroups v1 used a separate hierarchy per controller, which made consistent policy across resources awkward and produced a long tail of edge cases. cgroups v2 uses a single unified hierarchy and is now effectively universal on current distributions; Podman 6.0 dropped v1 support entirely, and Kubernetes has moved the same way. If you encounter cgroups v1 in 2026, you are on an old system and should expect tooling friction.

The failure mode worth knowing is the memory limit. When a container exceeds its cgroup memory limit, the kernel's out-of-memory (OOM) killer terminates a process inside it — usually the largest, which is usually your application. From inside the container this looks like an unexplained hard kill: no stack trace, no exception, no log line, exit code 137. From the host it is an entry in the kernel log. This is the single most common cause of containers that "just restart for no reason", and it is why a JVM or Python process with a heap sized from the host's total memory rather than the cgroup limit will die under load. Modern runtimes read cgroup limits, but only if configured to, and older ones do not.

CPU limits fail more subtly. The CPU controller in bandwidth mode gives a container a quota per scheduling period; when the quota is exhausted, every thread in the container is stopped until the next period begins. For a latency-sensitive multi-threaded service this produces periodic stalls that look like network problems and do not show up as high CPU

utilisation, because the process is not running when it is throttled. Throttling metrics exist in the cgroup filesystem and are worth watching precisely because the obvious metrics hide the problem.

2.4 Union Filesystems and the Image Layer Model

The third primitive is the layered filesystem. A container image is a stack of read-only layers, each a tarball of filesystem changes relative to the layer below. At runtime, a union filesystem — OverlayFS on essentially all current Linux systems — presents that stack as a single coherent directory tree, with a thin writable layer on top of it for the container's own changes.

Reads resolve down the stack: the topmost layer containing a given path wins. Writes go to the writable layer, and a write to a file that exists in a lower layer triggers a copy-up, where the whole file is copied into the writable layer first. Deletions are recorded as *whiteout* entries, special markers in the upper layer that mask a path present below.

This design buys three things. Images share layers, so ten images built from the same base image store that base once on disk and pull it once over the network. Builds cache, because a build step whose inputs haven't changed can reuse the layer it produced last time. And container startup is nearly free, because starting a container means creating a writable layer and mounting an overlay, not copying a filesystem.

The design also produces specific, recurring problems. Copy-up means the first write to a large file in a lower layer is slow and consumes disk equal to the file's size — which is why persistent database data should use a volume or external storage rather than the container's writable layer. Deleted files still occupy space in the layer where they were added, so a Dockerfile that downloads a 500MB archive in one `RUN` and deletes it in the next produces an image containing both the archive and the whiteout; the fix is to do both in a single layer, or to use a multi-stage build and copy only the result. And because layer identity is content-addressed, changing anything early in a Dockerfile invalidates every layer after it — the reason the conventional ordering puts dependency installation before application code.

2.5 Container Versus Virtual Machine

The comparison is worth doing precisely, because the differences follow from one architectural fact rather than being a list of unrelated tradeoffs.

A virtual machine virtualises hardware. A hypervisor — KVM, Xen, Hyper-V — presents virtual CPUs, memory, and devices to a guest, which boots its own kernel and runs its own

userspace. The isolation boundary is the hypervisor, a small and heavily-scrutinised piece of code with a narrow interface. Escaping it requires a hypervisor vulnerability.

A container shares the host kernel. The isolation boundary is the kernel's namespace and cgroup implementation, reached through the entire system call interface — hundreds of calls, each with its own argument-validation code. Escaping it requires a kernel vulnerability, and the kernel is a vastly larger attack surface than a hypervisor.

Everything else follows. Containers start in milliseconds because there is no kernel to boot and no hardware to enumerate; VMs take seconds. Containers do not need a second kernel: the processes inside are host processes, while VMs pay for a guest kernel and its memory. Containers must share the host's kernel version and its Linux-ness, so a Linux container cannot run on a Windows kernel — which is why Docker Desktop on macOS and Windows runs a Linux VM and puts your containers inside it, a detail that explains most of the performance and filesystem-mount weirdness on those platforms. VMs can run any guest OS.

And the security asymmetry is real, not theoretical. Container escapes via kernel vulnerabilities have happened repeatedly. If your threat model includes hostile code — multi-tenant workloads, customer-supplied builds, or, increasingly in 2026, AI agents executing generated code — plain namespace isolation is not enough on its own. That gap is exactly what gVisor, Kata Containers, and Firecracker exist to fill, and they are covered later.

3 The Practical Container Contract

The kernel primitives explain how containers are isolated, but they do not explain the interfaces you use every day. A container image, a container's writable state, its network attachment, and its security profile are separate pieces. Keeping them separate prevents several common category errors — treating an image as a running container, treating a published port as a service-discovery mechanism, or treating a non-root process as a complete security boundary.

3.1 Building and Identifying Images

An image is a portable filesystem and configuration description; a container is one runtime instance of it. A Dockerfile or Containerfile describes a build, but the file itself is not the image and the image is not the source repository. The build context — the files made available to the builder — is part of the input, which is why a careless context can make builds slow and can accidentally copy secrets into an image. `.dockerignore` or its equivalent is a correctness

and security control, not just an optimisation.

Use multi-stage builds when the build toolchain is larger or more privileged than the runtime needs. The final stage should contain the application, its runtime dependencies, and the minimum operating-system material it needs, not compilers, package caches, credentials, or test fixtures. BuildKit and Buildah provide different implementations of this workflow; the resulting OCI image is the interoperable artefact.

An image tag is a human-friendly pointer, not an immutable identity. `example/api:latest` can refer to different bytes tomorrow. A digest such as `example/api@sha256:...` identifies one manifest, and a multi-platform tag points to an image index that selects an architecture-specific manifest at pull time. Use tags for discovery and digests for promotion, rollback, and auditability. A build that is reproducible in intent but deploys a moving tag is not reproducible in practice.

3.2 Storage and Lifecycle

The writable layer described above is disposable container state. It disappears when the container is removed and is a poor place for data that must survive replacement. A named volume gives the engine ownership of persistent storage; a bind mount exposes a host path directly; a `tmpfs` mount keeps temporary data in memory. These choices have different ownership, backup, portability, and security consequences, especially in rootless mode. A mount is not baked back into the image when you commit or rebuild the container.

This is also why containers should be replaceable rather than patched in place. Put application code and immutable dependencies in the image, configuration in the environment or mounted files, and durable state in a volume or external service. The model is simple; the difficult part is deciding explicitly which data belongs in which category.

3.3 Networking Is a Separate Layer

A network namespace gives a container its own interfaces, routes, and port space; it does not, by itself, connect the container to anything. Engines usually attach it to a bridge through a virtual Ethernet pair, provide container-local DNS, and install forwarding and address-translation rules. Publishing a port maps a host port to a container port. It is not the same as making the container discoverable by name to other containers, and neither operation is required when containers share a network namespace in a Pod.

The common modes are worth distinguishing. A bridge network gives ordinary isolation with explicit connectivity. `host` networking removes the network namespace and shares the host's

ports, trading isolation for simplicity or performance. `none` gives the container no configured network. Kubernetes builds on the same primitives but imposes a different contract: the Pod, not each individual container, is the network endpoint.

3.4 Security Beyond Namespaces

Namespaces and cgroups are necessary but not sufficient security controls. Linux capabilities split root's powers into individual privileges; a container that does not need `NET_ADMIN` or `SYS_ADMIN` should not have them. Seccomp filters restrict which system calls a process may make. SELinux and AppArmor apply mandatory access-control policy outside the process's own view. Read-only root filesystems, dropped capabilities, and no-new-privileges reduce the consequences of a compromise.

The dangerous shortcuts are `--privileged`, mounting sensitive host paths, and exposing a root-equivalent engine socket. Rootless containers improve the default blast radius by mapping container UID 0 to an unprivileged host identity, but they still share the host kernel and still need a threat model. For hostile code, use a stronger boundary such as a sandboxed runtime or a microVM, and treat network egress and credentials as part of the isolation problem too.

4 Standardisation: The OCI Specifications

By 2015 the container ecosystem was heading for a format war. Docker had the momentum, but CoreOS had launched a competing runtime, rkt, with its own image specification, `appc`, on the explicit argument that a single vendor should not own the format. A split would have been genuinely damaging: images built for one engine would not have run on the other, registries would have had to support both, and every tool would have needed two code paths.

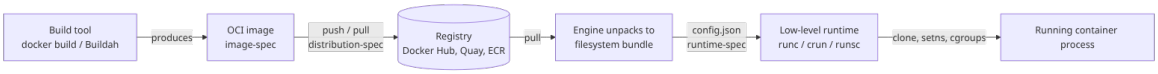
The resolution was the Open Container Initiative, a Linux Foundation body founded in 2015 with Docker donating its image format and runtime as the starting point. The OCI produces three specifications, and understanding what each covers explains most of the ecosystem's shape.

The **image specification** defines what a container image *is* as a data structure: a set of layer blobs, a manifest listing them with their digests, a configuration object holding the default command, environment, working directory and metadata, and an optional index for multi-platform images. Everything is content-addressed by cryptographic digest, which makes images immutable by construction and verifiable by anyone.

The **runtime specification** defines what a low-level runtime consumes: a *filesystem bundle*,

which is a directory containing an extracted root filesystem and a `config.json` describing how to run it — which namespaces to create, which cgroup limits to apply, what to mount, which capabilities to retain, what process to execute. The runtime spec is deliberately narrow. It says nothing about images, registries, or networking; it describes the act of turning one bundle into one running container, and the lifecycle operations — create, start, kill, delete — on it.

The **distribution specification** defines the registry HTTP API: how a client discovers, pulls, and pushes manifests and blobs. It was standardised later than the other two, formalising the Docker Registry v2 protocol that had already become universal in practice.



The separation is the point. Because the boundaries are specified rather than implied by one implementation, each stage can be replaced independently. An image built by Buildah runs under `containerd`. An image pulled from GitHub's registry runs under CRI-O. A `config.json` produced by Podman can be executed by `crun` instead of `runc` by changing one configuration line.

Two practical consequences are worth drawing out. First, Podman's claim of Docker compatibility is not reverse engineering; both consume and produce the same specified formats, so compatibility is the default and divergence is the exception. Second, when Kubernetes removed `dockershim` in version 1.24 in 2022 — the adapter layer that let the kubelet talk to Docker specifically — the widespread panic about "Kubernetes dropping Docker support" was misplaced. Kubernetes dropped a Docker-specific *integration*, not Docker-built *images*, because images are an OCI concern and always had been. Clusters switched their runtime to `containerd` or CRI-O and kept running the same images. Standardisation is what made that a configuration change rather than a migration.

The one meaningful gap in this otherwise tidy picture is the build step. Nothing in the OCI specifies how an image is *produced*; the Dockerfile format is a de facto standard maintained by Docker rather than a specified one. That is why build tooling is where the most divergence remains, and why Cloud Native Buildpacks — which produce OCI images from source without a Dockerfile — are a genuinely interesting development rather than yet another build wrapper.

5 Docker

Docker is where almost everyone's container knowledge starts, and its architecture is more layered than its single friendly CLI suggests. Knowing the layers matters because failures

land in specific ones, and because the layers are what the rest of the ecosystem reuses.

5.1 Origins and What Docker Actually Invented

Docker began as an internal tool at dotCloud, a platform-as-a-service company, and was open-sourced in March 2013; dotCloud subsequently renamed itself Docker Inc. and sold off the PaaS business. The initial implementation was a wrapper around LXC, later replaced by Docker's own `libcontainer`, which eventually became `runc`.

What Docker invented was the interface, not the isolation. Three specific things drove adoption. The `Dockerfile` made image construction declarative and reproducible in a format a developer could read in thirty seconds. The layered image format with content-addressed layers made images cheap to store and fast to distribute. And Docker Hub gave the world a default place to publish and find them, so `docker run nginx` worked on a fresh machine with no setup. That combination turned a kernel capability into a workflow, and the workflow is what spread.

5.2 Architecture: CLI, dockerd, containerd, runc

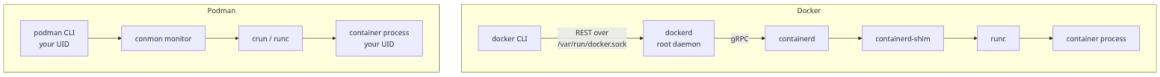
Docker is a client-server system, and the server side has been progressively decomposed.

The `docker` CLI is a thin client. It speaks HTTP over a Unix socket — `/var/run/docker.sock` by default — to **dockerd**, the Docker daemon. Everything you do with the CLI is an API call; the CLI itself holds no state and does no container work.

`dockerd` is a long-running process that, in the default configuration, runs as root. It owns the high-level concerns: the image cache, network and volume management, build orchestration, the Compose integration, and the API surface. It does not create containers itself.

Instead it delegates to **containerd**, a daemon that manages the container lifecycle at a lower level: pulling and unpacking images, managing snapshots, and supervising containers. `containerd` was extracted from Docker and donated to the CNCF, where it is a graduated project, and it is now used well beyond Docker — it is the default runtime for most Kubernetes distributions.

`containerd` in turn does not create containers directly either. For each container it starts a **shim** process, and the shim invokes **runc**, the OCI low-level runtime, which does the actual work: `clone()` with the right namespace flags, cgroup setup, mount configuration, capability dropping, and finally `execve()` of your process. `runc` then exits — it is not a supervisor. The shim remains as the container's parent, which is what allows `containerd` or `dockerd` to be restarted without killing running containers.



The decomposition has a real benefit and a real cost. The benefit is reuse: `containerd` and `runc` became shared infrastructure for the whole industry, including Kubernetes, which is why the ecosystem is not duplicating this work five times. The cost is that `dockerd` remains a single root-privileged daemon that everything flows through, and that has two consequences worth taking seriously.

The first is availability. If `dockerd` dies or is wedged, you lose the API — no `docker ps`, no `docker logs`, no ability to start or stop anything — even though the containers themselves, parented to their shims, keep running. Recovering means restarting the daemon, and a daemon restart with the wrong `live-restore` configuration takes your containers with it.

The second is security, and it is the more important one. Access to the Docker socket is equivalent to root on the host. Not "similar to" — equivalent. Anyone who can talk to `/var/run/docker.sock` can ask the daemon to run a container that bind-mounts the host root filesystem with privileges, and from there do anything. This is why adding a user to the `docker` group is a privilege grant, not a convenience, and why mounting the Docker socket into a container — a common pattern for CI runners and for tools that need to build images — hands that container full host control. Docker offers a rootless mode that addresses this, but it is opt-in and requires additional setup, and the default remains the root daemon. That asymmetry is the core of the Podman argument later.

5.3 Licensing and Commercial Status in 2026

This has become a genuine procurement question rather than a licensing footnote, and it is one of the two real drivers of engine choice in 2026.

Docker Inc. tightened its Subscription Service Agreement such that free use of **Docker Desktop** — the packaged GUI and VM-based product for macOS, Windows, and Linux — is limited to non-commercial and personal use, open-source work, and commercial use only by organisations with **fewer than 250 employees and under $10 million in annual revenue**. Cross either threshold and a paid subscription is required for every user. Government entities are excluded from the free tier regardless of size.

The paid tiers change over time and bundle different features. The relevant editorial point is that commercial Docker Desktop use is tied to a subscription, with organisation plans adding controls such as single sign-on, provisioning, audit features, and hardened isolation. Check Docker's current plan and subscription pages for prices, seat thresholds, and regulated-

industry terms rather than copying a number into procurement paperwork.

Two clarifications matter and are routinely confused. First, this is a licence on **Docker Desktop**, not on the Docker Engine or the container format — the engine remains open source under Apache 2.0, and running `dockerd` on a Linux server costs nothing. The bite lands on developer laptops, especially macOS and Windows fleets where Desktop is the practical way to get a Linux container host. Second, Docker Desktop's commercial terms are a product decision, not a change to OCI image compatibility or to the licensing of every container image you pull.

The practical effect is that a 400-person engineering organisation now has a five-figure-plus annual line item where it previously had none, and finance departments have started asking whether an Apache-2.0 alternative would do. That question is why Podman evaluations spiked, and it is a better predictor of engine choice in 2026 than any feature comparison.

5.4 Docker Compose

Compose remains the standard tool for running multi-container applications locally, and it is genuinely good at the job. A `compose.yaml` file declares a set of services, each with an image or build context, plus networks, volumes, environment, and dependency ordering; `docker compose up` brings the whole set up on a single host with a shared network where services resolve each other by name.

Two things to get right. First, the modern implementation is a Docker CLI plugin invoked as `docker compose` — the old standalone Python tool invoked as `docker-compose` is the v1 implementation and is retired. Copy-pasted commands with the hyphen are a reliable sign of a stale tutorial. Second, the Compose file format is now a published specification, which is why Podman and other tools can consume the same files.

Compose's boundary is that it is a single-host tool. It has no scheduler, no notion of a fleet, and no self-healing beyond restart policies. Using it in production on one machine is defensible for small deployments and widely done; expecting it to scale across hosts is a category error, and the attempts to stretch it in that direction are what Swarm and Kubernetes exist to replace.

5.5 Docker Swarm

Swarm is Docker's built-in clustering and orchestration mode, integrated into the engine since Docker 1.12 in 2016. It lets a set of Docker hosts form a cluster with managers and workers, and deploy services across them with a familiar CLI and a Compose-like file format. It does

scheduling, rolling updates, an overlay network, and service discovery, and for a small cluster it is dramatically simpler to stand up and reason about than Kubernetes.

Its current status needs stating carefully, because both "Swarm is dead" and "Swarm is a viable Kubernetes alternative" are wrong. It remains usable and receives maintenance, but it is not where the ecosystem's new orchestration work is happening. That means less current documentation, fewer integrations, and a smaller hiring pool than Kubernetes. Treat it as a deliberately small and stable platform choice, not as the default path for a new multi-team platform.

The adoption gap is stark. CNCF's January 2026 survey put Kubernetes production adoption among container users at 82%, against roughly 2.5% for Docker Swarm — the same survey cited later in this primer for the Kubernetes-versus-ECS comparison, so treat both figures with the same weight. That ratio, not the maintenance status, is the decisive fact: choosing Swarm means accepting a small pool of practitioners, documentation, tooling, and hiring candidates. It remains a reasonable choice for a genuinely small, stable deployment run by a team that does not want to operate Kubernetes — and a poor choice for anything expected to grow.

5.6 Docker Hub

Docker Hub is still the default and largest public registry, and it is where the overwhelming majority of base images and official images live. It is also a recurring operational irritant, principally because of pull rate limits on anonymous and free-tier accounts.

The failure pattern is specific and worth recognising because it is so common. A CI pipeline runs from cloud infrastructure, where many tenants share a small pool of outbound NAT addresses. Docker Hub rate-limits anonymous pulls by source IP. Your build starts failing with `toomanyrequests` on a pull that has worked for a year, intermittently, with no change on your side. The remedies are authenticating pulls, running a pull-through cache, or — most durably — mirroring the base images you depend on into a registry you control.

That last option is now standard practice at any scale, and it is why GitHub Container Registry, Amazon ECR, Google Artifact Registry, and Quay all have serious adoption. The distribution specification is what makes moving between them a configuration change rather than a rewrite, and Skopeo, covered shortly, is the tool that makes mirroring straightforward.

6 Podman

Podman is an OCI-compliant container engine developed primarily by Red Hat, designed around two architectural decisions that differ from Docker's: it has no daemon, and it runs rootless by default. It is also deliberately CLI-compatible with Docker, to the point that `alias docker=podman` works for the large majority of everyday commands. The name comes from "pod manager", which is a hint about a feature discussed below.

6.1 Daemonless Architecture

When you run `podman run`, there is no daemon. The `podman` binary does the work in your own process: it resolves the image, unpacks it, constructs the OCI bundle and `config.json`, and invokes the low-level runtime — `crun` by default — which creates the container. A small monitoring process called `conmon` is retained per container to hold its terminal, handle logging, and report the exit status. When the container exits, `conmon` exits. Nothing long-running persists.

The consequences are mostly good. There is no single process whose failure takes out your whole container API, because there is no central process at all; state lives in files under your home directory. There is no root-owned socket that is equivalent to host root, because there is no socket by default. Containers are children of the invoking process, so normal Unix tooling applies — `systemd` can supervise a container the way it supervises any other service, signals propagate as you would expect, and process accounting works.

The costs are real too. Without a daemon there is nothing to restart your containers after a reboot, so persistence is delegated to `systemd` rather than being built in — which is fine, and arguably better, but it is a different workflow that you have to know about. Tooling that expects to talk to a Docker API socket needs Podman's optional REST API service enabled, which reintroduces a socket, though it can run as a user service rather than as root. And image storage is per-user by default, so an image pulled by your user is not visible to another user or to a root-run container, which surprises people the first time.

6.2 Rootless by Default, and Why It Matters

This is the substantive security argument, and it rests on the user namespace.

In rootless mode, the container runs under your own UID. The user namespace maps UIDs inside the container to different UIDs outside it, using ranges allocated to your user in `/etc/subuid` and `/etc/subgid`. Root inside the container — user ID (UID) 0 — maps to an unprivileged host UID somewhere in your assigned range. Software inside the container

sees itself as root and behaves normally: it can bind low ports within its own namespace, write to `/etc`, install packages. From the host's point of view, every one of those actions is performed by an unprivileged user with no special capabilities.

The security property that buys you is specific and valuable. If an attacker breaks out of the container, they land as an unprivileged host user, not as root. They have not escalated; they have moved sideways into a low-privilege account. Compare that with the default Docker path, where a container escape or a socket compromise lands on a root daemon.

It is worth being precise about what this does *not* do, because overselling it is common. Rootless containers still share the host kernel, so a kernel vulnerability reachable from an unprivileged user is still a path to escalation. User namespaces themselves have had vulnerabilities. Rootless mode reduces blast radius substantially; it does not make containers a hard security boundary. For that you need the sandboxed runtimes discussed later.

Rootless mode also has friction that you will meet in practice. Volume mounts are the main one: a file owned by your host UID appears inside the container owned by the mapped UID, which frequently is not the UID your application runs as, and the result is permission errors that look inexplicable until you know about the mapping. Podman's `:U` mount option and `podman unshare` exist to deal with this. Binding privileged host ports below 1024 requires either a sysctl change or port mapping above the threshold. Rootless networking historically went through a userspace proxy with a performance cost, which is one of the things `netavark` and `pasta` improved.

Docker does support rootless mode, and it works. The difference is defaults and therefore practice: Podman is rootless unless you ask otherwise, Docker is rooted unless you configure otherwise, and defaults determine what the overwhelming majority of installations actually run. In a regulated environment where you have to evidence that no workload runs under a root daemon, that asymmetry is the whole argument.

6.3 systemd Integration and Quadlet

Podman's `systemd` integration is its most underrated feature and the thing that most changes how you deploy on a single host. Because containers are ordinary child processes rather than daemon-managed objects, `systemd` can supervise them directly.

Quadlet is the mechanism. You write a declarative unit file — `.container`, `.pod`, `.volume`, `.network`, or `.kube` — describing the container, drop it in a `systemd` unit directory, and Quadlet generates a proper service unit from it at boot. The container then behaves like any other system service: `systemctl start`, `systemctl status`, dependency ordering, restart

policies, journal integration, and user-level services via `systemctl --user` with lingering enabled.

```
[Unit]
Description=Application API service

[Container]
Image=registry.example.com/app/api:1.4.2
PublishPort=8080:8080
Volume=api-data.volume:/var/lib/app
Environment=LOG_LEVEL=info

[Service]
Restart=always

[Install]
WantedBy=default.target
```

That covers a surprisingly large class of deployments where people reach for Kubernetes without needing it. If you are running a handful of services on one or two machines and your real requirements are "start on boot, restart on failure, log to the journal, have dependencies ordered correctly", `systemd` already does all of that well, and Quadlet lets containers participate. Reaching for a cluster orchestrator to get restart-on-failure is a large amount of accidental complexity for a feature `systemd` has had for over a decade.

6.4 Compose, Desktop, and Kubernetes YAML

Podman's `podman compose` command is a thin wrapper around an external Compose provider, such as Docker Compose (`docker-compose`) or `podman-compose`. The provider implements Compose; Podman sets up the environment and passes the command through. Docker Compose can communicate with Podman's API socket, while `podman-compose` translates Compose files into Podman operations. If both providers are installed, `docker-compose` takes precedence by default; select one explicitly with `PODMAN_COMPOSE_PROVIDER` or `compose_providers` in `containers.conf`. Compatibility depends on the provider and workload, so test your Compose file with the provider you will use, especially when it relies on Docker-specific behaviour.

Podman Desktop is Red Hat's graphical equivalent to Docker Desktop — container and image management, Compose support, Kubernetes integration, and extensions — and it is actively developed. It is the answer to the licensing question for organisations that need a GUI developer experience on macOS or Windows without a per-seat subscription.

Podman also does something Docker does not: `podman generate kube` emits Kubernetes YAML for a running container or pod, and `podman play kube` runs a Kubernetes manifest locally. Neither is a substitute for a real cluster, and the generated YAML needs review

rather than direct deployment, but as a way to move between local development and cluster deployment without hand-translating, it is genuinely useful.

6.5 Podman's "Pod" Concept

Podman has a native grouping primitive called a **pod**: a set of containers that share network and IPC namespaces, so they reach each other on `localhost`, plus an infrastructure container that holds the shared namespaces open while individual containers come and go.

Kubernetes' Pod predates this by several years — the object goes back to Kubernetes' original 2014–2015 design, itself derived from Google Borg's internal grouping concept. Podman, which grew out of Red Hat's CRI-O work in 2017–2018, deliberately adopted the same shared-namespace grouping so that a local Podman workflow would interoperate cleanly with real Kubernetes manifests: `podman generate kube` and `podman play kube` only make sense as a round trip if the concept is the same on both sides. Podman is named for it — "Pod Manager". The practical value is that the mental model transfers: if you understand why a sidecar container in a Podman pod can talk to the main container on `localhost`, you understand Kubernetes Pod networking, because it is the same mechanism, adopted on purpose rather than reinvented in parallel.

6.6 Buildah and Skopeo

Podman ships as part of a set of tools that split responsibilities Docker combines into one daemon, which is a meaningfully different design philosophy rather than just packaging.

Buildah builds images. It consumes Dockerfiles, so it is a drop-in for `docker build`, but it also exposes image construction as a scriptable sequence of shell commands — create a working container from a base image, run commands in it, copy files in, set configuration, commit it to an image. That matters for two reasons. It lets you build images in a shell script with full control instead of squeezing logic into Dockerfile syntax, and it lets you build images without a full container runtime or any privileged daemon, which is what makes unprivileged in-cluster image builds practical. Podman's own `podman build` is Buildah embedded as a library.

Skopeo moves and inspects images without running them. `skopeo inspect` reads a remote image's manifest and configuration without pulling the layers, which is the cheap way to check a tag's digest or a base image's labels. `skopeo copy` transfers images directly between registries, or between a registry and a local directory or archive, without an intermediate local pull-and-push. Two jobs make this essential: mirroring upstream images into an internal

registry, and moving images into air-gapped environments via a tarball. Skopeo also handles signing and signature verification.

The separation is the point. Docker's model is one daemon that builds, stores, distributes, and runs. Podman's model is separate tools for separate concerns, each usable unprivileged, each composable in a script. For interactive development the difference is small. For automation and for CI systems that should not have privileged access, it is substantial.

6.7 Governance, Versions, and the CNCF Sandbox

Podman has always been Red Hat-led, and single-vendor governance was a legitimate objection to adopting it — the mirror image of the objection to Docker.

In January 2025, the Podman Container Tools project — Podman, Buildah, and Skopeo — entered the Cloud Native Computing Foundation Sandbox. That is meaningful external stewardship, but it is not CNCF graduation or proof of fully neutral governance: Red Hat remains the primary sponsor and contributor. Podman Desktop should be assessed separately. For organisations whose adoption criteria include foundation participation, the move reduces — but does not eliminate — the single-vendor concern.

Version-sensitive details belong in release notes rather than in a durable mental-model primer. If you are upgrading across a major Podman release, check the migration notes for storage backends, cgroups, networking, and rootless connectivity — especially if you have custom CNI configuration or scripts that depend on implementation details.

6.8 Adoption Reality

Here the honest framing matters more than the advocacy. Podman's architecture is better on the axes that platform engineers and security teams care about, and its usage is nowhere near Docker's.

Podman's real installed base comes substantially from being the default engine on RHEL-family distributions and Fedora, where it arrives without anyone choosing it. Outside that ecosystem, Docker remains the default assumption in tutorials, CI templates, tooling defaults, and job descriptions. The exact market-share numbers are less useful than that distinction, and survey populations vary too much to support false precision here.

So: Podman is the more defensible architecture, winning on paper and dominant within the Red Hat ecosystem, and not winning overall market share. Both halves of that are true, and a recommendation that ignores either is incomplete.

7 Docker Versus Podman in 2026

Most comparisons of these two tools that you will find online were written between 2019 and 2021, and they are wrong now in a specific way: they are organised around feature gaps that no longer exist. Podman lacked Compose support, lacked a Desktop application, lacked Docker API compatibility, had rough networking. All of those were true and all of them have been addressed. If you are making this decision in 2026 on the basis of a feature matrix from 2021, you are answering a question that has changed.

7.1 The Technical Differences That Remain

Dimension	Docker	Podman
Architecture	Client-server via `dockerd`, root by default	Daemonless; direct fork-exec, `conmon` per container
Privilege model	Root daemon by default; rootless is opt-in configuration	Rootless by default with user-namespace UID mapping
Socket exposure	`/var/run/docker.sock` is root-equivalent	No socket by default; optional user-level API service
Default low-level runtime	Commonly `runc`; configurable	Commonly `crun`; configurable
`systemd` integration	Bolt-on, awkward	First-class via Quadlet unit files
CLI and image compatibility	Native	Near drop-in for the Docker CLI and Compose files
Build tooling	`docker build` in the daemon; BuildKit	Buildah, usable standalone and unprivileged
Licence (commercial use)	Desktop subscription terms apply; verify current thresholds	Open-source components; no Docker Desktop-style seat licence
Governance	Docker Inc., single vendor; `containerd` and `runc` in the CNCF	Podman Container Tools in the CNCF Sandbox; Red Hat remains primary sponsor

Dimension	Docker	Podman
Ecosystem position	Dominant; Docker Hub; largest tooling and tutorial base	Minority overall; default on RHEL-family distributions

The rows that actually decide things are the licence row and the privilege row. The rest are differences you would adapt to within a week.

7.2 When to Pick Docker

Pick Docker when compatibility with the surrounding world is the dominant constraint. Every tutorial, CI template, IDE integration, testing library, and Stack Overflow answer assumes Docker. Testcontainers, devcontainers, and a long list of language-specific tooling target the Docker API first, and while most of it works against Podman's compatible socket, "most" means you will occasionally be the one debugging the exception.

Pick Docker when your organisation is already invested and the switching cost is real — retraining, rewriting internal tooling, revalidating CI. That cost is not enormous, but it is not zero, and a migration needs a reason beyond architectural preference.

Pick Docker when your developers are on macOS or Windows and Docker Desktop's polish matters to you, and either you are under the free-tier thresholds or you are willing to pay. The subscription is not expensive per seat; the question is whether you are getting anything for it that the alternative does not provide.

7.3 When to Pick Podman

Pick Podman when eliminating the root daemon is a requirement rather than a preference. In regulated environments — financial services, healthcare, government — "no workload runs under a root-privileged daemon" is the kind of control that is easy to evidence with Podman and awkward with Docker. The same applies to shared build hosts and CI runners, where the alternative is handing every job root-equivalent socket access.

Pick Podman when you are on RHEL, Fedora, CentOS Stream, or AlmaLinux. It is the supported default, it is what the distribution's documentation and support assume, and installing Docker there means running against the grain of the platform for no benefit.

Pick Podman when the Docker Desktop licence is a genuine cost you would rather not carry. For a several-hundred-developer organisation this is a straightforward financial calculation,

and Podman Desktop is a credible replacement rather than a concession.

Pick Podman when you want containers as `systemd` services on a single host. Quadlet is materially better than anything Docker offers for this, and for a large class of small deployments it is the right answer instead of a cluster orchestrator.

7.4 The Honest Summary

For greenfield Linux work with no platform constraint, Podman is the better default in 2026: same images, near-identical CLI, a safer default privilege model, and no Docker Desktop seat licence. The reason not to choose it is ecosystem gravity, and ecosystem gravity is a real engineering force, not a failure of nerve — being the only team in the company using a different engine has an ongoing cost.

Also worth saying plainly: this choice matters less than the amount written about it suggests. Both produce OCI images that run anywhere. Both consume the same Dockerfiles. In production on Kubernetes neither is present — the cluster runs `containerd` or CRI-O, and your build engine is a development and CI concern only. The decision is about developer workflow and build-time privilege, not about what runs in production, and that scoping should keep it proportionate.

8 The Wider Runtime Landscape

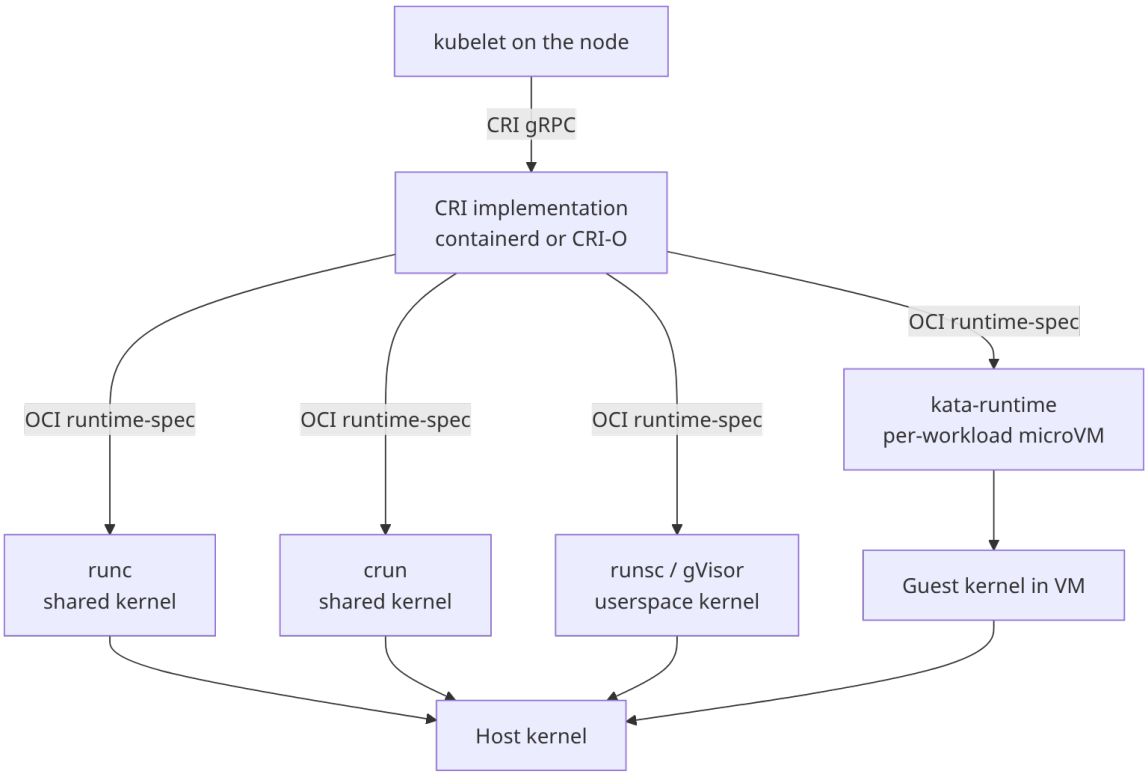
Docker and Podman are *engines* — user-facing tools that manage images and orchestrate the act of running a container. Beneath and beside them sits a set of components that matter once you touch Kubernetes or have isolation requirements that plain containers cannot meet. The most important distinction to get straight is the one between a CRI implementation and an OCI runtime, because the terms are used interchangeably in casual writing and they are not interchangeable at all.

A **CRI implementation** talks upwards to Kubernetes. The Container Runtime Interface is a gRPC API that the kubelet uses to say "pull this image", "create a sandbox for this Pod", "start this container". `containerd` with its CRI plugin and CRI-O are the two common implementations.

An **OCI runtime** talks downwards to the kernel. It consumes a filesystem bundle and a `config.json` and performs the actual isolation work. `runc`, `crun`, `runsc` (gVisor), and `kata-runtime` are OCI runtimes.

They sit in a stack, and multiple OCI runtimes can be available under a single CRI imple-

mentation, selectable per workload.



8.1 containerd

`containerd` is the mid-level runtime that manages the container lifecycle: pulling and storing images, managing filesystem snapshots, creating and supervising containers through shims, and handling low-level networking hooks. It was extracted from Docker, donated to the CNCF, and has since graduated — the CNCF's highest maturity tier, requiring demonstrated production adoption, a diverse contributor base, and a security audit.

Its significance is that it became shared infrastructure. Docker uses it. Most Kubernetes distributions use it with its CRI plugin, including the managed offerings from the major clouds. That convergence is healthy: the component doing the most operationally sensitive work is one well-audited, widely-deployed implementation rather than five.

8.2 CRI-O

CRI-O is a CRI implementation built for exactly one purpose: running OCI containers under Kubernetes, with nothing else in scope. No build support, no CLI for interactive use, no features that Kubernetes does not require. It is Red Hat-backed and is the runtime in

OpenShift.

The argument for it is minimality. A component with a smaller feature surface has a smaller attack surface and fewer ways to behave unexpectedly, and CRI-O's release cadence is tied to Kubernetes' own, so version compatibility is unambiguous. The argument against is ecosystem size — `containerd` has broader adoption and consequently more tooling that assumes it. For most practitioners this is not a decision you make; it comes with your distribution.

8.3 runc and crun

`runc` is the OCI reference implementation, written in Go, extracted from Docker's `libcontainer`. It remains one of the most widely deployed container runtimes and is a common default in `containerd`; current CRI-O configurations commonly default to `crun` instead. When you read the runtime specification and wonder what a compliant implementation looks like, `runc` is the canonical reference point.

`crun` is a smaller alternative written in C, developed principally by Red Hat, and it is Podman's default. The advantages follow from the language and the smaller scope: lower memory footprint per container, faster startup — Red Hat has reported around 22% faster container start, which is a vendor figure and should be read as "meaningfully but not dramatically faster" — and no Go runtime in the process. `crun` also gained cgroups v2 support earlier than `runc` did, which is how it became Podman's default.

The practical relevance is at density. If you are starting thousands of short-lived containers, per-container overhead compounds and `crun` is worth benchmarking. For a dozen long-running services the difference is invisible. Both are OCI-compliant and interchangeable via configuration, so this is a tuning decision rather than an architectural one.

8.4 gVisor

Standard containers share the host kernel, and that is the security ceiling described earlier. gVisor, from Google, addresses it by inserting a kernel of its own.

gVisor's `runsc` runtime intercepts the container's system calls and services them in a userspace kernel called the Sentry, written in Go, which reimplements a substantial portion of the Linux system call interface. The container's syscalls do not reach the host kernel directly; they reach the Sentry, which makes a much narrower set of calls to the host on the container's behalf. The attack surface presented to hostile code shrinks from the full Linux syscall interface to the Sentry's implementation plus whatever the Sentry itself needs from the host.

The tradeoffs are concrete. You pay a syscall performance penalty, which ranges from negligible for compute-bound work to significant for syscall-heavy or I/O-heavy workloads. Compatibility is good but not total — gVisor implements most of Linux, and applications relying on obscure syscalls or particular `/proc` behaviour can break. In return you get isolation considerably stronger than namespaces without requiring hardware virtualisation, which means it works in nested-virtualisation environments where a VM-based approach cannot.

AI agent sandboxing is a new demand driver for gVisor. Running code generated by a language model is, from a security standpoint, running untrusted code submitted by an anonymous party. The exact adoption growth figures are vendor datapoints and do not belong in a durable primer; the useful point is that agent execution makes the shared-kernel threat model concrete for teams that previously ran only trusted application code.

8.5 Kata Containers

Kata takes the other route: if the problem is a shared kernel, give each workload its own. Kata runs each container or Pod inside a lightweight virtual machine via KVM or Cloud Hypervisor, with its own guest kernel, while presenting a standard OCI runtime interface so Kubernetes treats it as a runtime choice rather than a different deployment model.

The isolation guarantee is therefore the hypervisor's, which is the strongest boundary available short of separate physical machines. The cost is higher than gVisor's: a guest kernel per workload consumes memory, boot time is longer than a container's though far shorter than a traditional VM's, and hardware virtualisation must be available — which rules it out inside VMs that do not expose nested virtualisation, a common constraint on some cloud instance types.

Kata's position in 2026 is genuine multi-tenant Kubernetes: clusters running workloads from mutually untrusting parties, where a container escape crossing tenant boundaries is an unacceptable outcome and the memory overhead is an acceptable price.

8.6 Choosing Between Them, and RuntimeClass

The 2026 framing worth carrying is that these are no longer cluster-wide decisions. Kubernetes' **RuntimeClass** object lets you define named runtime configurations and select one per Pod, so a single cluster can run trusted internal services on `runc`, customer-submitted code on gVisor, and multi-tenant workloads on Kata. That per-workload selection is the significant operational change, because it removes the need to pay isolation overhead uniformly or to run separate clusters per trust level.

Roughly: Firecracker is common in serverless and AI-sandbox platforms, Kata is a strong fit for multi-tenant Kubernetes, and gVisor is a strong fit for workloads that need a userspace kernel without a full microVM. All three can be consumed as workload-level isolation choices rather than as platform-wide commitments.

8.7 LXC and LXD

LXC and LXD are frequently filed as "alternatives to Docker", which misses that they answer a different question.

Docker and Podman are **application container** tools. The model is one primary process per container, an immutable image, ephemeral instances, and upgrade-by-replacement: you do not patch a running container, you build a new image and replace it. Configuration comes from environment variables and mounted files; state lives in volumes or external services.

LXC and LXD are **system container** tools. The model is a persistent, stateful, multi-process operating system environment that behaves like a lightweight VM. It boots an init system, runs `sshd` and `cron` and a syslog daemon, you log into it, you apply package updates in place, and it has a lifespan measured in months. It is a machine, managed like a machine, that happens to share the host kernel rather than being virtualised.

LXC is the low-level toolset and predates Docker — Docker's first releases used LXC underneath. LXD is the higher-level management daemon with a REST API, clustering, live migration, and storage and network management, originally from Canonical; note that a fork called Incus emerged after LXD's governance moved under Canonical's CLA, and both are actively maintained, so you will encounter either name.

The reason to care is that the system container model is right for some problems. Consolidating a fleet of lightly-loaded Linux VMs onto fewer physical hosts, providing developers with persistent long-lived Linux environments, or running software that genuinely expects a full OS with multiple services and an init system are all cases where forcing the application container model produces an awkward result. "Use LXD" is a legitimate answer that the container discourse tends to forget exists.

8.8 WebAssembly and WASI

WebAssembly — Wasm — is a portable binary instruction format originally built for running compiled code in browsers. WASI, the WebAssembly System Interface, extends it outside the browser by defining a capability-based interface to system resources: files, sockets, clocks, randomness. Together they make a deployment target: compile your code to a Wasm module,

run it in any compliant runtime on any platform.

The genuine advantages are real. Cold starts are in the single-digit to low-tens-of-milliseconds range against hundreds of milliseconds for a container, because there is no filesystem to mount and no process to fork — just a module to instantiate. Module sizes are dramatically smaller than container images, commonly by one to two orders of magnitude, since there is no base OS. Portability is by architecture rather than by convention: the same module runs on x86 and ARM without a multi-platform build. And the security model is capability-based and deny-by-default, so a module has no filesystem or network access unless explicitly granted — which is a stronger starting position than a container's.

The state in 2026 is that Wasm is genuinely maturing rather than perpetually promising. WASI Preview 2 and the Component Model are important parts of the current direction, but support still varies by runtime and host platform. Check the target platform's compatibility rather than assuming that a Wasm module is as portable operationally as its binary format suggests.

Now the correction, because it matters. **Wasm is not replacing containers, and framing that says it is should be read as marketing.** The "Docker is dead, Wasm killed it" genre is clickbait. The reasons are structural, not maturity-related. Wasm modules are single-language compiled artefacts with a constrained system interface; containers package arbitrary software including things you did not write and cannot recompile. Wasm's threading, filesystem, and networking support remains narrower than a full OS. Anything stateful, anything with a complex dependency tree, anything involving a database or an existing binary you do not control, is a container workload and will stay one.

Where Wasm wins is where its properties match the problem: edge compute, where cold start and module size dominate; serverless functions, where the same is true; and plugin and extension systems, where running untrusted third-party code inside a host application is exactly what a capability-sandboxed module is for. That last category may end up being the largest, and it has no container equivalent at all.

You will see specific adoption statistics quoted for Wasm — figures in the 60-to-70-percent range for developer evaluation or new enterprise projects. Treat those as soft: they originate largely from vendor-sponsored surveys with self-selected respondents, and they do not withstand the scrutiny you would apply to a CNCF survey. The defensible statement is that Wasm has real production adoption in edge and plugin contexts, is improving quickly, and is complementary to containers. Hybrid deployment is the emerging norm; replacement is not.

8.9 Firecracker and the microVM Model

Firecracker is an open-source virtual machine monitor written in Rust by AWS, purpose-built for serverless. It is the technology under AWS Lambda and Fargate, which makes it one of the most heavily-exercised pieces of virtualisation software in existence.

Its design is aggressive minimalism. A general-purpose VMM like QEMU emulates a wide range of devices to support arbitrary guest operating systems. Firecracker emulates a deliberately tiny set — virtio block and network devices, a serial console, a keyboard controller with one key for reset — and nothing else. The published characteristics follow: boot to userspace in around 125 milliseconds, under 5 MiB of memory overhead per microVM, and up to roughly 150 microVMs created per second on a single host.

That combination is what makes VM-grade isolation viable for serverless. Every Lambda invocation environment is a separate microVM with its own kernel, which is why AWS can run code from unrelated customers on the same hardware with a hypervisor boundary between them, at a per-invocation cost that a conventional VM could never support.

The notable recent development is **Lambda MicroVMs**, launched in June 2026, which extends Lambda's maximum execution time from 15 minutes to 8 hours, with persistent state across the lifetime of an execution environment and snapshot-based boot. That is a substantial change to what Lambda is for — the 15-minute ceiling was the main reason long-running jobs, batch processing, and agent-style workloads had to go elsewhere. At launch it is limited to Graviton/Arm instances in a subset of regions, initially US-East, US-West, Tokyo, and Ireland, so check availability before designing around it.

For a practitioner, Firecracker mostly matters as infrastructure you consume rather than operate. But the pattern — VM-grade isolation at container-grade startup cost — is the one to keep in mind, because it is the direction the isolation story has been moving and it is what makes per-workload isolation choices affordable.

9 Use Cases: From One Container to a Platform

Containers solve a set of practical problems before they become an orchestration strategy. They package an application and its user-space dependencies into a portable artefact, give a job a predictable execution environment, and make replacement cheap. They do not make data durable, schedule work across a fleet, or turn a single process into a distributed system. Those boundaries matter particularly in data and machine learning work, where the code is often portable but the data, hardware, and coordination are not.

The useful progression is not "container, then Kubernetes" as a universal maturity model. It is a sequence of increasingly demanding operational problems:

Situation	What the container solves	What to add next
One developer or CI job	Dependency drift, native-library differences, and "works on my machine" failures	A pinned image, a clean build, and a registry
One batch job or service on one host	Repeatable execution and simple replacement	A volume or object store for state; `cron`, systemd, Quadlet, or a managed job runner
Several services on one host	Local networking, process grouping, and coordinated startup	Compose or Quadlet, backups, monitoring, and an explicit upgrade procedure
Many independent jobs or services	Portable workload units	A managed batch/container service, or a scheduler if placement and quota become a problem
A shared multi-node fleet	Placement, rescheduling, rollout, autoscaling, and multi-team policy	Kubernetes or another orchestrator, with the operational expertise to run it

The transition should be driven by coordination and failure requirements, not by a desire to appear production-grade. A container running a serious model on one machine can be entirely appropriate. A lightweight web service spread across thirty nodes may need orchestration even if its code is trivial.

9.1 Reproducible Development and CI

The most broadly useful container is often not a production container. A development image can pin the Python or R version, system libraries, compilers, command-line tools, and GPU user-space libraries that a project expects. A CI job can then build the same image, run tests in a clean environment, and publish the exact artefact that will be promoted later. This removes a large class of failures caused by different host distributions or a developer's accumulated local packages.

For a data team, that means a notebook environment can match the batch job and the model-serving image closely enough that a transformation or import does not change behaviour between stages. Mount the repository and datasets needed for interactive work; keep the environment and its dependencies in the image. Do not put credentials, large mutable datasets, or notebook state in the image. Use image digests for promotion, because a mutable tag makes a supposedly reproducible pipeline ambiguous.

This use case needs no server, cluster, or persistent container. A local engine and a CI runner are enough. The container is valuable because it is a contract for the environment, not because it is long-lived.

9.2 Batch Data Work

An ETL, feature-generation, validation, or backfill job is a natural container workload. The image contains the code and runtime; input data comes from object storage, a warehouse, a database, or a mounted filesystem; output is written to a durable system. The job can be retried or run with a different image digest without modifying the host. A failed job leaves a diagnosable execution record rather than a half-updated machine.

Start with one container invoked by a scheduler, a managed batch service, or a workflow system. Add a queue when jobs compete for limited workers. Add parallel fan-out when one input can be partitioned into independent tasks. Add a cluster scheduler only when placement, fairness, retries, resource quotas, or the number of workers has become the problem. Kubernetes is one way to provide those controls; it is not required merely because the job processes terabytes.

The important separation is between compute and data. The image should not contain the dataset, and the container's writable layer should not be the system of record. Object storage and warehouses usually provide the durability and access patterns; the container provides a repeatable computation over them.

9.3 Notebooks and Interactive Data Science

Containers are useful for giving a team a shared starting environment without requiring everyone to install the same operating system packages. A notebook server can run in a container with the project dependencies, a mounted working directory, and access to a controlled data source. The image can be rebuilt when dependencies change, while notebooks and generated results live outside the disposable container.

There is a trap here: a notebook server is interactive and stateful even when its container

is not. Persist the notebook files and intentional outputs, and make long-running work a separately recorded job where possible. A container restart should not be treated as a backup strategy. For a small team, one host plus an access-control layer is often enough; a multi-user notebook platform earns Kubernetes only when it needs placement, per-user isolation, GPU allocation, idle reaping, and shared service integration at scale.

9.4 Machine Learning Training

An ML training image captures the framework, Python packages, compilers, tokenizers, and other user-space dependencies that otherwise make experiments difficult to reproduce. It can also include the CUDA or ROCm user-space libraries expected by the framework. The host still supplies the kernel, GPU device, and usually the vendor driver; the image does not magically make incompatible hardware compatible. GPU access requires an engine/runtime integration and a host configured to expose the device.

For one researcher with one GPU, a local engine or a single VM is usually the right level. For a team sharing a few machines, a queue or managed batch service can allocate GPUs, record logs, retry failed jobs, and prevent one experiment from consuming every device. Containers make those jobs portable; the queue solves contention.

Distributed training changes the shape of the problem. A job may need several GPUs at once, topology-aware placement, coordinated startup, shared checkpoints, and an all-or-nothing allocation so that it does not occupy half the cluster while waiting for the rest. That is where gang scheduling, resource quotas, GPU partitioning, and a cluster scheduler become valuable. Kubernetes can provide the substrate, but the default scheduler alone does not provide the complete ML platform; systems such as Kueue, Volcano, device plugins or Dynamic Resource Allocation, and higher-level frameworks each address different parts of the problem.

9.5 Model Serving and Inference

Serving a model is another ordinary container use case at small scale: package the model server and its dependencies, mount or fetch the model artefact, expose an HTTP or gRPC endpoint, and replace the image when the model or server changes. Readiness checks, resource limits, and an external model registry matter more than the choice of engine.

As serving grows, the requirements become operational. Different models may need different CPU, memory, or GPU profiles; traffic may be bursty; revisions need gradual rollout and rollback; and some models can scale to zero while others need warm capacity. A managed container or serverless inference platform can supply those controls. Kubernetes becomes

compelling when many teams need a shared, policy-controlled fleet with heterogeneous hardware and workload-specific scaling. It is not automatically the best answer for one model endpoint.

9.6 When the Ladder Reaches Kubernetes

Kubernetes earns its complexity when several independent demands arrive together: workloads no longer fit one host; nodes fail; services need stable discovery; deployments must roll out without downtime; teams need quotas and access boundaries; and jobs compete for specialised resources. Data and ML platforms often reach this point through shared GPU capacity, distributed training, multi-tenant notebook or inference services, and a mixture of batch and online workloads.

Even then, use a managed Kubernetes service when the organisation needs Kubernetes' API and ecosystem but does not need to own the control plane. Use a managed Spark, Ray, batch, or ML platform when it expresses the workload more directly. The right abstraction is the smallest one that handles the coordination problem: a container packages the work, a scheduler places it, and a platform adds policy and lifecycle management only where those capabilities are needed.

10 Why Kubernetes Exists

Kubernetes has a reputation for complexity that it largely deserves, and the standard explanation — that it is complex because it is powerful, or popular because it is popular — explains nothing. The useful explanation is that Kubernetes is a set of answers to specific operational problems, and each piece of its complexity traces back to one of them. If you do not have those problems, you do not need the machinery that solves them, and the honest version of this section is as much about when Kubernetes is unnecessary as about why it exists.

The problems all appear at the same moment: when you have more containers than fit comfortably on one machine, across more than one machine, and you need them to keep working without someone watching.

10.1 Scheduling Across a Fleet

With one host, placement is not a question — the container runs here. With thirty hosts it is the central question, and it is a bin-packing problem: each workload needs some amount of CPU and memory, each node has a finite amount, and you want high utilisation without overcommitting any node into resource starvation. Doing this by hand means maintaining

a mental or spreadsheet model of what runs where, and updating it every time anything changes.

Kubernetes inverts the relationship. You declare what a workload needs — requests, and optionally limits — and the scheduler decides where it goes, continuously, using current cluster state. You never name a machine. That inversion is the single largest conceptual shift from managing hosts to managing a cluster, and it is what makes the rest possible: if you never specified a machine, nothing breaks when a machine goes away.

10.2 Self-Healing

Processes crash. Nodes fail, get rebooted for kernel patches, run out of disk, or lose network connectivity. On a single host, `systemd` or a restart policy handles a crashed process, and that covers a lot. Neither handles a dead node — the containers that were on it are gone, and something has to notice and place them elsewhere.

Kubernetes handles this through continuous reconciliation rather than event handling, which is a distinction worth dwelling on. There is no "node failure handler" that fires on an event. There is a controller that perpetually compares the number of running replicas against the declared number and acts on any difference, whatever caused it. A crashed container, a drained node, a deleted Pod, and an evicted workload all produce the same observation — fewer replicas than declared — and the same response. That is why the system is robust to failure modes nobody enumerated in advance.

10.3 Declarative Reconciliation

This is the architectural idea that everything else in Kubernetes is built on, and it is the one worth understanding properly.

Imperative management means issuing commands: run this container, stop that one, scale this to five. Each command takes effect once. If reality drifts afterwards — something crashes, someone makes a manual change, a node disappears — nothing corrects it, because the command has already completed. Configuration management tools partially address this by re-running periodically, but they act on hosts, not on a fleet-wide model of intent.

Declarative management means submitting desired state: there should be three replicas of this image, with this configuration, exposed on this port. Controllers then run continuous loops that observe actual state, compare it to desired state, and take action to close the gap. The loop never terminates. Drift is not a special case to be detected; it is the normal input to a loop that is always running.

The practical consequences are large. Your entire cluster configuration becomes a set of declarative documents that can live in Git, be reviewed, be diffed, and be reapplied idempotently — which is the entire foundation of GitOps. Rollback becomes reapplying a previous document. And the system self-corrects continuously rather than only when someone notices.

10.4 Service Discovery and Stable Networking

Containers get new IP addresses when they are rescheduled, and in a system that reschedules continuously, every IP address is temporary. Hard-coding one is guaranteed to break. Maintaining a load balancer configuration by hand against a constantly-changing backend set is not sustainable past a handful of services.

Kubernetes provides stable virtual addresses and DNS names that resolve to the current set of healthy Pods behind them, updated automatically as Pods come and go. Your application connects to `payments-api` and does not know or care which Pods exist or where. This is unglamorous and it is one of the things you would most miss building this yourself.

10.5 Rolling Updates and Rollbacks

Deploying a new version across many replicas without downtime requires a coordinated sequence: start new instances, wait for them to report healthy, shift traffic, retire old instances, and stop and reverse the whole thing if the new version fails its health checks. Every step has failure modes, and doing it by hand across twenty replicas is both tedious and error-prone at exactly the moment when errors are most expensive.

Kubernetes Deployments encode this. You change the image tag; the controller performs the rollout according to a configured strategy, respecting constraints on how many replicas may be unavailable and how many extra may exist during the transition, gated on readiness probes. It also retains previous versions, so rollback is one command. The important detail is the readiness gate: a new replica receives no traffic until it says it is ready, which is what makes a bad deployment stall rather than cascade — provided your readiness probe actually tests something meaningful, which is a common and consequential failure to get right.

10.6 Scaling

Manual scaling in Kubernetes is changing a number and letting reconciliation do the rest. Automatic scaling comes in layers: the Horizontal Pod Autoscaler adds and removes replicas based on observed CPU, memory, or custom metrics; the Vertical Pod Autoscaler adjusts the resource requests of individual Pods; and a cluster autoscaler adds and removes nodes when

Pods cannot be scheduled or nodes sit idle. The three compose, which is how a cluster can grow from ten to a hundred nodes under load and shrink back without intervention.

10.7 Multi-Tenancy and Resource Governance

Once a cluster is shared, you need boundaries. Namespaces partition the cluster logically, so names do not collide and access can be scoped. Role-based access control (RBAC) governs who may do what to which resources. ResourceQuotas cap total consumption per namespace, and LimitRanges constrain individual workloads, which together prevent one team's runaway job from starving everyone else. NetworkPolicies restrict which workloads may talk to which.

These mechanisms are how one cluster serves many teams rather than each team operating its own. That consolidation is a large part of Kubernetes' economic argument, and it is also the part most often implemented badly — soft multi-tenancy via namespaces is not a security boundary against hostile tenants, which is precisely why Kata Containers exists.

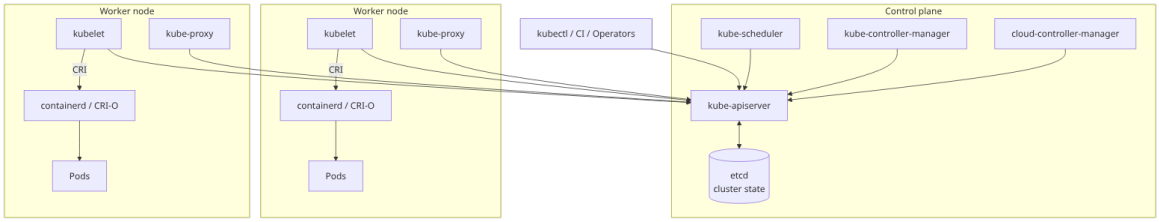
10.8 The Honest Counter-Case

Every capability above is genuine, and none of it is free. You are taking on a distributed system with its own failure modes, a large object model to learn, a version upgrade treadmill, and a networking and storage layer that requires real expertise when it misbehaves. The common failure is not choosing Kubernetes when you should not have; it is choosing it and then not investing enough to operate it competently, which leaves you with all the complexity and none of the reliability.

The threshold question is straightforward: do you actually have the coordination problem? If you run four services on two machines, `systemd` with Podman Quadlet or a managed container service will serve you better, and you will spend your time on your product rather than on your platform. If you run sixty services across thirty nodes with multiple teams deploying daily, you need something that does what Kubernetes does, and building it yourself is the worse option. The section on alternatives returns to this with concrete options.

11 Kubernetes Core Architecture

Kubernetes is a control system. Its architecture divides into a **control plane** that holds desired state and makes decisions, and **nodes** that run workloads and report status. The API server is the hub for cluster state and control-plane coordination; node agents also communicate with their local runtime, network, and storage components. That distinction is the key to understanding both how the system behaves and how it fails.



11.1 kube-apiserver

The API server is the front door and the only component that reads or writes cluster state. `kubectl` talks to it. Controllers talk to it. The kubelet on every node talks to it. Operators, CI pipelines, and dashboards talk to it. It exposes a REST API, authenticates and authorises every request, runs admission control on mutations, validates objects against their schemas, and persists them to `etcd`.

Two properties follow from this design. First, it is horizontally scalable and stateless — you run several instances behind a load balancer for availability, since all state is in `etcd`. Second, it is the single point of failure for cluster *management*, though not for running workloads: with the API server down, you cannot deploy, scale, or inspect anything, but existing Pods continue running because the kubelet keeps them going from its local view. A control plane outage is a management outage, not necessarily a service outage, and knowing that distinction is useful during an incident.

The other mechanism to know is **watch**. Clients do not poll; they open a long-lived watch on a resource type and receive a stream of change events. Every controller in the system is built on this, which is what makes reconciliation responsive rather than a periodic sweep, and it is also why a controller with a badly-written watch loop can generate enormous API server load.

11.2 etcd

`etcd` is a distributed key-value store using the Raft consensus algorithm, and it holds all cluster state — every object, its specification, and its status. It is the cluster's source of truth, and the cluster is exactly as durable as `etcd` is.

The operational facts that matter: it needs an odd number of members, typically three or five, to tolerate failures while maintaining quorum; it is extremely sensitive to disk write latency and network latency between members, because every write requires a quorum `fsync`, which is why `etcd` on slow or network-attached storage produces a cluster that feels inexplicably sluggish; and losing quorum means the cluster becomes read-only until it is restored. Backups

are non-negotiable, because restoring `etcd` is restoring the cluster and there is no other copy of that state.

One more thing worth knowing: Secrets are stored in `etcd` and, unless encryption at rest is explicitly configured, they are stored merely base64-encoded. Anyone with `etcd` access or an `etcd` backup has every Secret in the cluster in plaintext.

11.3 kube-scheduler

The scheduler watches for Pods that have no node assigned and assigns them. Its algorithm runs in two phases: **filtering**, which eliminates nodes that cannot run the Pod — insufficient free resources against its requests, unsatisfied node selectors or affinity rules, taints the Pod does not tolerate, unavailable volumes — and **scoring**, which ranks the surviving nodes by a set of weighted priorities such as spreading replicas across nodes and balancing resource usage. The highest-scoring node wins, and the scheduler writes the assignment back to the API server. It does not start anything; the kubelet on the chosen node notices the assignment and acts.

The distinction between **requests** and **limits** is important. A request is the amount the scheduler reserves when placing the Pod; a limit is a runtime ceiling enforced by the kubelet and cgroups. A Pod can be scheduled because its requests fit and still be throttled or killed when it reaches its limits. Leaving both fields unset does not mean "unlimited and safe" — it means the workload is relying on namespace and node defaults, and may be able to consume resources opportunistically until another workload needs them.

The failure mode you will meet is a Pod stuck in `Pending`, which almost always means filtering eliminated every node. The usual causes are resource requests larger than any node's allocatable capacity, node selectors or affinity rules that match nothing, taints without matching tolerations, and a PersistentVolumeClaim that cannot be bound. `kubectl describe pod` reports the scheduler's reasoning in the events, and reading that output is the fastest diagnostic in Kubernetes.

The scheduler is also where the AI-workload story breaks down, because it schedules Pods one at a time and independently — which is exactly wrong for a distributed training job that needs sixteen GPUs simultaneously or not at all. That gap is what Volcano and Kueue exist to fill, covered in the trends section.

11.4 kube-controller-manager

The controller manager is a single binary running many independent control loops, each responsible for one kind of reconciliation. The Deployment controller manages ReplicaSets. The ReplicaSet controller manages Pods. The node controller watches node health and marks nodes unreachable. The Job controller tracks run-to-completion workloads. The endpoint controller maintains the Pod lists behind Services. There are dozens.

Every one follows the same shape: watch the relevant objects, compare observed state to desired state, act to reduce the difference, repeat. That uniformity is why Kubernetes is extensible in a coherent way — a custom Operator you write is architecturally identical to the built-in controllers, using the same API, the same watch mechanism, and the same reconciliation pattern. There is no privileged extension mechanism because controllers were never privileged to begin with.

A separate **cloud-controller-manager** holds the loops that talk to a cloud provider's APIs — provisioning load balancers for Services of type `LoadBalancer`, attaching block storage, and managing node lifecycle against the provider's instance model. Splitting it out is what allowed provider-specific code to leave the Kubernetes core.

11.5 kubelet

The kubelet is the agent on every node, and it is the component that actually makes containers exist. It watches the API server for Pods assigned to its node, and for each one it instructs the container runtime through the CRI to create the Pod sandbox and start the containers. It then continuously reports status back: whether containers are running, their restart counts, the node's capacity and conditions.

The kubelet also runs **probes**, and getting these right is one of the highest-leverage things you can do. A **liveness** probe determines whether a container is healthy; on failure the kubelet restarts it. A **readiness** probe determines whether it should receive traffic; on failure it is removed from Service endpoints but not restarted. A **startup** probe gives slow-starting applications time before the liveness probe begins, which is what prevents a slow-booting application from being killed in a restart loop.

The classic disaster is a liveness probe that checks a dependency. If your liveness endpoint queries the database, then a database outage makes every replica fail liveness, so the kubelet restarts all of them simultaneously, so none is available even after the database recovers, and you have converted a dependency degradation into a total outage of your own making. Liveness should test only whether this process is functioning; readiness is where dependency

checks belong.

11.6 kube-proxy and the Networking Model

`kube-proxy` runs on each node and implements Service networking: it watches Services and their endpoints and programs the node's packet-handling rules so that traffic to a Service's virtual IP is load-balanced to a healthy backing Pod. Implementations include `iptables` mode, IPVS mode for better performance with many Services, and eBPF-based dataplanes from CNI plugins such as Cilium that can replace `kube-proxy` entirely.

The underlying networking model is worth stating because it is assumed everywhere: every Pod gets its own IP address, every Pod can reach every other Pod's IP directly without network address translation, and nodes can reach all Pods. Kubernetes does not implement this — it requires it, and a **CNI** (Container Network Interface) plugin such as Calico, Cilium, or Flannel provides it. That is why cluster networking behaviour varies between clusters: the model is specified, the implementation is pluggable, and the observable behaviour — particularly around NetworkPolicy enforcement and performance — depends on which plugin you have.

12 The Kubernetes Object Model

Kubernetes' objects are often presented as a glossary, which is the least useful possible framing. Each object exists because a specific problem needed a first-class representation, and knowing the problem tells you when to use it.

12.1 Pod

The Pod is the smallest thing Kubernetes schedules. It is one or more containers that share a network namespace, an IPC namespace, and any volumes you define, and that are always placed on the same node and scheduled as a unit.

Why is the unit a Pod rather than a container? Some containers are inseparable. A log-shipping agent that reads files your application writes needs the same filesystem and the same lifecycle. A proxy handling inbound TLS for your application needs the same network namespace so it can forward to `localhost`. Scheduling those independently would be wrong in every case. The Pod exists so that "these must be co-located and share resources" is expressible.

This does not mean you should put your application and its database in one Pod. The test is whether the containers must share a lifecycle and a machine; if they can be scaled, updated,

or placed independently, they belong in separate Pods. Pods also have **init containers**, which run to completion in sequence before the main containers start — the right place for schema migrations, waiting on a dependency, or fetching configuration.

The critical property is that Pods are disposable. They are never repaired; they are deleted and replaced. A Pod's name and IP address are valid only for that instance. Every other workload object exists because Pods are ephemeral and something has to manage their replacement.

12.2 ReplicaSet and Deployment

A **ReplicaSet** keeps a specified number of identical Pods running: it watches Pods matching its selector, creates more when there are too few, deletes some when there are too many. That is its entire job, and it is the self-healing mechanism in concrete form.

You almost never create one directly, because a ReplicaSet cannot change the Pod template of running Pods — it has no concept of a version transition. **Deployment** sits above it and adds exactly that. A Deployment owns a sequence of ReplicaSets; when you change the Pod template, it creates a new ReplicaSet and shifts replicas from old to new according to a rollout strategy, respecting `maxUnavailable` and `maxSurge` and gating on readiness. It keeps the old ReplicaSets scaled to zero as revision history, which is what makes `kubectl rollout undo` work.

Deployment is the right object for stateless workloads, which is most application code: any replica can serve any request, replicas are interchangeable, and replacing one loses nothing.

12.3 Service

Pods come and go with new IPs. A **Service** provides a stable name and address in front of a changing set of them. It selects Pods by label, and the endpoint controller keeps the list of healthy matching Pod IPs current; traffic to the Service is load-balanced across that list.

The types matter. `ClusterIP`, the default, gives an internal-only virtual IP and a DNS name — the standard way services inside a cluster reach each other. `NodePort` additionally opens a port on every node, which is crude but occasionally useful. `LoadBalancer` asks the cloud provider for an external load balancer pointing at the Service, which is how you expose something to the internet on a managed cluster. `ExternalName` is just a DNS alias to an outside host, useful for pointing at a managed database without embedding its hostname in application code.

Two details catch people out. Service load-balancing is at layer 4, so a long-lived connection

such as a gRPC stream is balanced once at connection time and then pinned — which is why gRPC traffic distributes badly across replicas without a layer-7 proxy or a mesh. And `ClusterIP` Services are reachable from every Pod in the cluster by default; restricting that requires NetworkPolicies.

12.4 Ingress and the Gateway API

A `LoadBalancer` Service per HTTP application means a cloud load balancer per application, which is expensive and gives you no shared routing layer. **Ingress** solves this: it is a set of HTTP routing rules — host, path, TLS certificate, backend Service — that a single shared entry point implements.

Ingress is only a specification of intent. Nothing happens unless an **ingress controller** is running to read those objects and configure an actual proxy. NGINX Ingress, Traefik, HAProxy, and cloud-provider controllers all do this, and their behaviour differs, which is why Ingress accumulated a large surface of controller-specific annotations for anything beyond basic routing — a design weakness that the **Gateway API** exists to correct. Gateway API is a richer, role-separated, properly-typed successor that expresses traffic splitting, header manipulation, and non-HTTP protocols as first-class fields rather than annotations, and separates cluster-operator concerns from application-team concerns. It is the direction of travel; Ingress will remain widely deployed for years, but new work should evaluate Gateway API.

12.5 ConfigMap and Secret

ConfigMap holds non-sensitive configuration as key-value pairs, consumable as environment variables or mounted as files. Its purpose is to get configuration out of the image, so that the same image runs in development, staging, and production with different settings — the prerequisite for promoting an artefact rather than rebuilding per environment.

Secret is structurally the same for sensitive data, and its default protections are weaker than most people assume. Secret values are **base64-encoded, not encrypted**. Base64 is an encoding, not a cipher. Without explicitly enabling encryption at rest, Secrets sit in `etcd` in effectively plaintext, and anyone with API access to read Secrets in a namespace, or access to an `etcd` backup, has them. Treating `kind: Secret` as sufficient protection is a common and serious mistake. Real practice means enabling encryption at rest, restricting read access through RBAC, and considering an external secret manager — Vault, or a cloud provider's Key Management Service (KMS)-backed store — with a CSI driver or operator. Check the integration carefully: some designs mount secrets without creating a Kubernetes

Secret, while others synchronise them into `etcd`.

One practical note: a Secret or ConfigMap mounted as a volume updates in place when the object changes, while one injected as an environment variable does not — environment variables are set at process start and cannot change. Applications expecting to pick up rotated credentials need the volume form and code that re-reads the file.

12.6 Namespace

A **Namespace** is a logical partition of the cluster. Object names are unique within a namespace, not across the cluster, and namespaces are the unit that RBAC rules, ResourceQuotas, and often NetworkPolicies are scoped to. They are the mechanism for letting several teams or environments share one cluster.

The limit is important: a Namespace is an administrative boundary, not a security boundary. Pods in different namespaces share the node kernel, can reach each other over the network unless a NetworkPolicy forbids it, and compete for the same node resources. Namespace-based separation is fine for teams within one organisation. It is not adequate isolation for mutually hostile tenants, which needs either separate clusters or sandboxed runtimes.

12.7 StatefulSet

Deployments treat replicas as interchangeable, which is wrong for stateful systems. A database replica has an identity: it is the primary or a specific follower, it owns a particular volume, and other members address it by name. **StatefulSet** provides that.

It gives each replica a stable ordinal identity — `db-0`, `db-1`, `db-2` — that survives rescheduling; a stable DNS name per replica via a headless Service, so members can address each other individually; a PersistentVolumeClaim per replica that follows it when it is rescheduled; and ordered, sequential creation, scaling, and rolling updates, so `db-1` is not started until `db-0` is ready. Those guarantees are what clustered databases, message brokers, and consensus systems require.

StatefulSet handles identity and storage. It does not handle application-level operations — leader election, failover, backup, safe version upgrades with schema changes — which is precisely why complex stateful software on Kubernetes is packaged as an Operator that uses a StatefulSet underneath rather than as a bare StatefulSet.

12.8 DaemonSet

A **DaemonSet** runs exactly one Pod on every node, or on every node matching a selector, and automatically places a Pod on any node that joins later. This is the shape of node-level infrastructure: log collectors that read the node's container logs, metrics agents that scrape node statistics, CNI plugin components, storage drivers, and security agents. The workload is per-machine rather than per-request, so replica counts are the wrong abstraction and node coverage is the right one.

12.9 Job and CronJob

A **Job** runs Pods to completion rather than continuously, tracking successful completions and retrying failures up to a backoff limit. It supports parallelism and a required completion count, which covers batch processing and work-queue patterns. **CronJob** creates Jobs on a schedule.

The behaviours to know: a Job's Pods are not cleaned up automatically unless you set a TTL, so a CronJob running every five minutes accumulates completed Pods until something removes them. And CronJob has no exactly-once guarantee — under control plane disruption you can get a missed run or a duplicate — so jobs must be idempotent. Concurrency policy controls whether an overrunning job blocks, replaces, or runs alongside the next scheduled one, and the default allows overlap, which surprises people whose jobs occasionally run long.

12.10 PersistentVolume and PersistentVolumeClaim

Container filesystems are ephemeral. Persistent storage in Kubernetes is deliberately split into two objects to separate two different concerns.

A **PersistentVolume** is a piece of actual storage in the cluster — a cloud disk, a Network File System (NFS) export, a logical unit number (LUN) — with a capacity, access modes, and a reclaim policy. It is an infrastructure object, typically the cluster administrator's concern.

A **PersistentVolumeClaim** is a request: an application asks for 20 GiB with a given access mode and storage class. Kubernetes binds the claim to a suitable volume, and the Pod references the claim, never the volume.

The separation means application manifests are portable — they ask for storage by class and size, not by cloud disk identifier — and it is what makes the same manifest work on different infrastructure. **StorageClass** completes the picture by enabling dynamic provisioning: rather than pre-creating volumes, a claim against a storage class causes a CSI driver to create the underlying storage on demand.

The constraint to internalise is access modes. `ReadWriteOnce` means the volume mounts read-write on one node, and it is what most block storage supports. `ReadWriteMany` means many nodes simultaneously, and requires a shared filesystem such as NFS or CephFS. A Deployment with multiple replicas sharing a `ReadWriteOnce` claim will schedule replicas onto one node or fail to schedule at all, and the resulting `Pending` Pods are one of the more confusing beginner failures. Block storage is also usually zone-bound, so a Pod with a claim can only be scheduled in the volume's zone — which quietly defeats multi-zone resilience if nobody notices.

13 The Kubernetes Ecosystem

Core Kubernetes gives you a control loop and an object model. Almost nobody runs it bare, because raw YAML at scale creates problems the core deliberately leaves out of scope.

13.1 Helm

Raw manifests do not parameterise. Deploying the same application to three environments means three near-identical directory trees differing in a handful of values, and installing third-party software means reading dozens of manifests and editing them by hand.

Helm is the de facto package manager. An application is packaged as a **chart**: templated manifests plus a `values.yaml` of defaults plus metadata. Installing produces a named **release** with a recorded revision, and upgrades and rollbacks operate on that release history. The public chart ecosystem is large, and for third-party infrastructure — an ingress controller, a monitoring stack, a database — `helm install` against a maintained chart is the normal path.

The recurring criticism is Go text templating over YAML, which means you are producing whitespace-sensitive structured data with a string templating engine. Indentation bugs, quoting bugs, and unreadable nested conditionals are routine, and debugging a chart often means running `helm template` and reading the rendered output. It is a genuine design weakness that the ecosystem tolerates because the packaging and release-management benefits are worth it.

13.2 Kustomize

Kustomize takes the opposite approach: no templating at all. You keep plain, valid Kubernetes manifests as a **base**, then define **overlays** that patch them — change the replica count, add an environment variable, set a different image tag, apply a name prefix. Overlays are

themselves valid YAML describing transformations, and the whole thing composes.

It has been built into `kubectl` since 1.14, so it needs nothing installed. The tradeoff against Helm is real and not merely stylistic: Kustomize's manifests are always valid YAML you can read and validate, which makes it far better suited to your own applications and to environment differentiation, but it has no packaging, no distribution, and no release history, so it does not replace Helm for consuming third-party software. Using Kustomize for your own manifests and Helm for other people's is a common and sensible arrangement. Helm charts can also be post-processed with Kustomize, which is how teams patch upstream charts without forking them.

13.3 Custom Resources and the Operator Pattern

Kubernetes' API is extensible. A **CustomResourceDefinition** registers a new object type with the API server — say, `PostgresCluster` — with its own schema, and from then on it behaves like a built-in type: `kubectl get postgresclusters` works, RBAC applies, and it is stored in `etcd`.

A CRD alone does nothing; it is a typed record. An **Operator** is a controller that watches those custom resources and reconciles reality to match them, encoding operational knowledge that would otherwise live in a runbook. A database operator knows how to provision a cluster with the requested replica count, elect a primary, take scheduled backups, fail over when the primary dies, and perform a version upgrade in the correct order with the right pre-flight checks. You declare `version: 16.2` and the operator performs the upgrade.

This is the single most important extension pattern in Kubernetes, and it is why complex stateful software runs on Kubernetes at all. The reason it works is the uniformity noted earlier: an Operator is architecturally identical to the built-in controllers. Frameworks — the Operator SDK, Kubebuilder, and Kopf for Python — handle the boilerplate.

The caution is that an Operator is production software running with broad cluster permissions, and its quality varies enormously. A mature, well-tested operator for a database is a substantial asset. A thinly-maintained one that mismanages a failover is a liability with cluster-admin rights. Evaluate them as you would any critical dependency.

13.4 Policy and Admission

Authentication answers who is making a request; authorisation answers whether that identity may make it; **admission** answers whether the requested object is acceptable before it is persisted. This is where a cluster can enforce rules such as "no privileged containers", "images

must come from this registry", "every workload needs resource requests", or "this namespace may use only these security contexts".

Kubernetes' built-in **Pod Security Admission** provides the baseline, restricted, and privileged Pod Security Standards at namespace scope. It is deliberately general. Teams that need organisation-specific rules commonly add a policy engine such as Kyverno or Gatekeeper, and supply-chain policy can use the same admission point to verify image signatures. Policy is part of the platform contract: test it against real workloads, report violations with enough detail to fix them, and keep an emergency path that is audited rather than relying on undocumented cluster-admin exceptions.

13.5 Observability and Debugging

Containers change where the useful evidence lives. Application output normally goes to stdout and stderr for the engine or kubelet to collect; the container's writable filesystem is not a durable log store. The runtime can tell you whether a process exited and with what status, the node can tell you about memory pressure and CPU throttling, and Kubernetes events can tell you why scheduling, mounting, or admission failed. None of those replaces application metrics and distributed traces, which explain behaviour after the process is technically running.

A practical diagnostic model follows the boundaries in this primer. First establish whether the image is the one you intended — inspect its digest and configuration. Then establish whether the process started and received the expected environment, mounts, and signals. Next check resource limits, network namespace and Service endpoints, readiness state, and node or volume conditions. In Kubernetes, `kubectl describe`, events, container logs, runtime metrics, and application telemetry answer different questions; reading only the application log is how a Pending Pod turns into a guessing exercise.

The operational requirement is centralisation. Logs, metrics, events, and traces should survive container and node replacement, carry workload identity, and be queryable across replicas. Otherwise the system's replacement model destroys the evidence needed to understand the failure it was designed to recover from.

13.6 Service Meshes and the Ambient Shift

A service mesh moves cross-cutting network concerns out of application code: mutual Transport Layer Security (TLS) between services, retries, timeouts, circuit breaking, traffic splitting for canary releases, and uniform request-level telemetry. Implementing those consistently across many services in several languages is the problem it solves.

The classic implementation was the **sidecar**: an Envoy proxy injected into every Pod, intercepting all traffic. It worked and it was expensive — a proxy per Pod costing memory and CPU, adding latency on both sides of every call, and coupling proxy upgrades to Pod restarts across the whole fleet.

The 2026 shift is away from that. **Istio Ambient Mesh** removes per-Pod sidecars in favour of a shared per-node component called `ztunnel` that handles layer-4 concerns — mutual TLS, identity, basic routing — for every Pod on the node, with optional per-service **waypoint** proxies deployed only where layer-7 features such as HTTP-level routing or policy are actually needed. Reported memory savings per workload versus sidecar Envoy exceed 90%, which is a vendor figure but directionally unsurprising given the architecture. Just as important operationally, upgrading the mesh no longer requires restarting every application Pod.

Linkerd remains positioned as the simpler alternative, with a purpose-built lightweight proxy, a much smaller configuration surface, and a reputation for being comprehensible. Benchmarks under load have reportedly still favoured it over Istio ambient in some cases; treat comparative mesh benchmarks with suspicion generally, since they are usually published by an interested party.

Practical guidance: if you are evaluating a mesh fresh in 2026, evaluate ambient architectures and do not start with sidecar Istio, which is the model being phased out. If your requirements are mutual TLS plus consistent observability without complex layer-7 policy, Linkerd is likely the lower-cost answer. And if you have a dozen services and no compliance requirement for service-to-service encryption, you may not need a mesh at all — it is a genuine increase in the number of things that can break in your request path.

13.7 GitOps: Argo CD and Flux

If desired state is declarative documents, those documents belong in version control, and something should continuously ensure the cluster matches them. That is **GitOps**: Git is the source of truth, and an in-cluster agent reconciles the cluster against the repository.

Argo CD and **Flux** are both CNCF graduated projects with strong production adoption. Argo CD is application-centric with a well-regarded web UI showing sync status and drift, which makes it easy to adopt and easy to explain to people who will not read YAML. Flux is a set of composable controllers; teams that want a UI typically add another tool, while teams that want GitOps as infrastructure rather than as a product often prefer Flux's controller model.

The properties that make this worth doing: every change is a reviewed commit with an audit

trail; the cluster self-corrects when someone makes a manual change, because the agent sees the drift and reverts it; recovery from cluster loss is applying the repository to a new cluster; and nobody needs cluster write credentials in CI, because the agent pulls rather than the pipeline pushing. That last point is a real security improvement and often the thing that sells it.

The discipline it demands is that manual changes may be reverted when reconciliation and pruning are configured to enforce the repository — which is the intended behaviour and is nonetheless disorienting for teams accustomed to `kubectl edit` during an incident.

14 Lightweight Kubernetes Distributions

Upstream Kubernetes assumes a certain scale: several control plane nodes, `etcd` on fast disks, a CNI plugin, a cloud provider integration. That is appropriate for a production cluster and absurd for a Raspberry Pi, a CI job, or a laptop. A family of lighter distributions exists for those cases, and they are not interchangeable — each optimises for something different, and picking the wrong one produces avoidable pain.

k3s, from Rancher and now SUSE, is the clear production-lightweight leader. It is fully CNCF-certified Kubernetes packaged as a single binary under 100MB that runs in around 512MB of RAM, with `etcd` replaced by default with an embedded SQLite-backed datastore (`etcd` remains available for high availability), and with a batteries-included set of defaults — a CNI plugin, an ingress controller, a load balancer implementation, and local storage — so a usable cluster comes up in one command. It works well air-gapped and on ARM. If you are deploying to edge locations, retail sites, industrial hardware, or running a small production cluster on a couple of machines, k3s is the right default.

k0s targets a similar niche with a different emphasis: zero host dependencies. It ships everything it needs in one binary and makes no assumptions about the host distribution, which matters when your fleet is a heterogeneous mixture of Linux versions you do not control.

MicroK8s, from Canonical, is distributed as a snap and is the best-integrated option on Ubuntu. It has self-healing high availability using `dqlite` and a clean add-on system for enabling components. If your infrastructure is Ubuntu throughout and you are comfortable with snaps, it is the path of least resistance.

kind — Kubernetes-in-Docker — runs cluster nodes as containers on a single host. Its purpose is testing: it creates and destroys multi-node clusters in seconds, which is exactly what a CI pipeline needs to run integration tests against a real API server. It is optimised

for that and is a poor choice for day-to-day development, since the container-in-container arrangement makes local image and volume workflows awkward.

minikube is the learning and local-development option: a single-node cluster with several driver choices, a built-in dashboard, and a mature add-on ecosystem covering ingress, metrics, and registries. It is the friendliest introduction and is not intended for CI or production.

The mental model: k3s, k0s, and MicroK8s are real if small-scale production, especially at the edge; kind is CI; minikube is learning and local development. All of them run the same API and the same manifests, which is the point — your manifests are portable across the whole range.

15 Alternatives to Kubernetes

Choosing Kubernetes should be a decision, not a default, and there are cases where it is the wrong answer.

HashiCorp Nomad is a single-binary cluster scheduler, considerably simpler to operate than Kubernetes and — unusually — not container-specific. It schedules containers, but also raw executables, Java applications, and virtual machines, using a pluggable task driver model. Its object model is much smaller, which means less to learn and less to misconfigure. Note the ownership change: HashiCorp's acquisition by IBM completed in February 2025, with the full operational transition in September 2025, and Nomad now sits within IBM's automation portfolio. Choose it when you want straightforward scheduling across mixed workload types without Kubernetes' object model, or when you are already invested in Consul and Vault. The cost is a much smaller ecosystem — there is no equivalent of the CNCF landscape or the operator ecosystem behind it.

Docker Swarm, covered earlier, still works and remains a defensible small-platform choice. It is a poor bet for anything that will grow or need to hire because its ecosystem and practitioner pool are much smaller than Kubernetes'.

Amazon ECS is AWS's own container orchestrator, and it is the most commonly underrated option. Its mental model is much smaller than Kubernetes' — task definitions and services, not a dozen object kinds — it integrates natively with IAM, CloudWatch, ALB, and the rest of AWS, and it has no control-plane charge, unlike EKS's per-cluster fee. On Fargate it removes node management entirely. The cost is total lock-in: an ECS task definition is not portable anywhere.

The trend data cuts both ways here. CNCF's January 2026 survey found 82% of container

users running Kubernetes in production, up from 66% in 2023 — a strong secular move towards Kubernetes even inside single-cloud shops. At the same time ECS remains heavily used, particularly for internal and batch workloads, and many enterprises run both: ECS where the workload is AWS-only and internal, EKS where portability or a Kubernetes-native ecosystem matters. The broader direction is consistent: avoiding node management can matter more than choosing a particular orchestrator.

When not to reach for full Kubernetes: when your team is too small to operate it competently, which is the most common failure and produces the worst outcomes; when you are single-cloud with no portability requirement and a managed service covers you; when your workloads are simpler than the object model assumes — a handful of stateless services with modest traffic; and when you are running on one or two machines, where Podman with Quadlet or Compose will do the job with a fraction of the moving parts.

16 Where Things Stand in 2026

A few developments from the past eighteen months are genuinely worth a practitioner knowing, as opposed to being conference noise.

16.1 Release Cadence and Versions

Kubernetes ships three minor releases a year. Version 1.37, codenamed "Garhwal", was released on 26 August 2026 with 67 enhancements — 16 graduating to stable, 23 to beta, 27 in alpha, and one deprecation. The preceding patch line, 1.36.4, came out on 11 August 2026. The practical consequence of the cadence is that each minor version receives about one year of patch support, so a cluster left alone for two years is out of support and its upgrade path is a multi-hop exercise. Upgrade planning is not optional work.

16.2 GPU Scheduling and AI Workloads

This is the biggest current storyline, and the underlying problem is the scheduler mismatch described earlier. Kubernetes schedules Pods independently, greedily, one at a time. A distributed training job needs sixteen GPUs simultaneously, on nodes with the right interconnect topology, or the allocated GPUs sit idle waiting for the rest — which at current GPU prices is expensive idleness. The exact share of cluster capacity used by AI workloads is hard to measure consistently; the operational pressure is clear even without a market-share number.

Dynamic Resource Allocation is the foundational change. DRA's core API graduated to

GA in Kubernetes 1.34, adding a richer path alongside the older device-plugin model for GPUs and other specialised hardware. Device plugins expose devices as opaque countable resources — "this node has 4 of `nvidia.com/gpu`" — while DRA models devices as claimable resources with attributes and structured parameters. That lets the scheduler reason about properties such as memory, topology, and device relationships without requiring every workload to understand the hardware layout. The two models will coexist while drivers and managed Kubernetes services catch up.

DRA is one layer of a stack that has stabilised around the following division of labour. **Kueue** handles admission and quota: it holds jobs in queues and admits them only when their full resource requirement is available, with hierarchical quotas and borrowing between teams — solving the problem of a cluster full of half-scheduled jobs that are all waiting and none running. **Volcano** provides batch and gang scheduling, where a job's Pods are scheduled all-or-nothing, plus fair-share and topology-aware placement. **HAMi** provides GPU sharing and virtualisation, letting several workloads share one physical GPU with memory and compute limits, which matters enormously for inference and interactive notebook workloads that cannot use a whole device. **NVIDIA MIG** does the same at the hardware level by partitioning a GPU into isolated instances. Above those, **Kubeflow** is the ML platform control plane, **KServe** provides serverless model serving with scale-to-zero, and **KubeRay** runs distributed Ray workloads.

The practical takeaway is that running serious AI workloads on Kubernetes means deliberately choosing components from this stack. The default scheduler does not by itself provide admission, gang scheduling, quota policy, GPU partitioning, or model-serving semantics, and discovering those gaps after committing to a cluster design is expensive.

16.3 Supply-Chain Security

Container supply-chain tooling has moved from advanced practice to baseline expectation, and the driver was a series of real incidents — the XZ Utils backdoor discovered in 2024 being the best-corroborated example of how far upstream a compromise can sit.

Sigstore is the centre of it. Its signing tool, **cosign**, signs container images and stores signatures in the registry alongside the image, so signature distribution is solved by the distribution specification you already use. The important innovation is **keyless signing**: rather than managing long-lived signing keys, a build system authenticates via OpenID Connect (OIDC) to Fulcio, which issues a short-lived certificate bound to that identity, and the signature plus its certificate is recorded in Rekor, a public transparency log. The result is a verifiable claim that a specific image was built by a specific workflow in a specific repository,

with no key material to steal or rotate.

This is moving from advanced practice towards baseline. Managed registries can store the signatures, and admission controllers can be configured to reject images without a valid signature from an expected identity. If your pipeline does not sign images and your cluster does not verify them, that is one of the most straightforward supply-chain improvements available.

16.4 Ecosystem Scale and the Wasm Correction

The CNCF landscape continues to grow, and Cloud Native Buildpacks are notable because they standardise producing OCI images from source without a Dockerfile. That is useful when you want a consistent build contract across languages, but it does not remove the need to understand the image, registry, and runtime boundaries described earlier.

That scale is a mixed signal. It reflects a genuinely healthy ecosystem and it also means the landscape diagram is no longer usable as a guide. Treat project count as evidence of activity, not as a shopping list; most organisations need a small, deliberately chosen subset, and the discipline of choosing few things and running them well beats breadth.

Two other threads already covered belong in any summary of 2026. Ambient service-mesh architectures are a serious alternative to sidecar-per-Pod deployments, not a universal replacement. And AI agent sandboxing is a new demand driver for gVisor, Kata, and Firecracker — isolating code execution for autonomous agents rather than relying only on classical multi-tenant SaaS patterns. That is a clear example of AI workloads reshaping infrastructure choices rather than merely consuming capacity.

17 Closing: A Usable Mental Model

If this primer leaves you with four things, let them be these.

A container is a process with a restricted view of the system, constrained resources, and a layered root filesystem. Everything else is tooling. When something behaves strangely, ask which namespace, which cgroup, or which layer is responsible, because the answer is almost always one of those three.

The OCI specifications are why the ecosystem is composable rather than a set of silos. Images, runtimes, and registries are separately specified, which is why Docker-built images run under Podman and `containerd`, why Kubernetes could drop `dockershim` without breaking anything, and why your engine choice does not determine your production runtime.

Docker versus Podman is a real decision with a clear default and low stakes. Podman is the better architecture — daemonless, rootless, Apache-licensed, with Podman Container Tools in the CNCF Sandbox — and Docker has overwhelming ecosystem gravity. Decide on licensing economics and security posture, not on 2021-era feature gaps, and remember that neither runs in your production cluster.

Kubernetes is a reconciliation engine, and that single idea explains all of it. You declare desired state; controllers observe actual state and act continuously to close the gap. Pods, Deployments, Services, and every custom resource are expressions of desired state, and every controller including the ones you write is the same loop. It is the right answer when you genuinely have the fleet coordination problem, and expensive accidental complexity when you do not — so answer that question honestly before you adopt it, because the cost of running it badly is higher than the cost of not running it at all.

18 Further Reading

These sources provide the specifications, current project documentation, and evidence behind the primer's time-sensitive claims.

- Open Container Initiative specifications — the image, runtime, and distribution standards that make engines and registries interoperable.
- Docker Desktop subscription terms — the current source for Docker Desktop's commercial conditions.
- Podman Container Tools in the CNCF — governance and project status for the Podman tool family.
- Kubernetes release history and support windows — release cadence and support information for version planning.
- Kubernetes 1.34: Dynamic Resource Allocation reaches GA — the primary explanation of DRA's GPU and specialised-resource model.
- CNCF Annual Cloud Native Survey, January 2026 — survey evidence for the adoption figures discussed in the primer.
- Kubernetes Pod Security Standards — the built-in baseline, restricted, and privileged policy profiles.
- Kubernetes observability overview — the official overview of metrics, logs, traces, and events.
- Sigstore documentation — signing, keyless identity, and transparency-log documentation for image supply chains.
- AWS Lambda MicroVMs announcement — the source for the Firecracker-related deployment example.